A Project Report on

PCB FAULT DETECTION

AI-DRIVEN OPTICAL PCB INSPECTION SYSTEM

Under subject of

SUMMER INTERNSHIP/SKILL BASED TRAINING (3170001)

of B. E. Semester – VII

Academic Year 2025-2026

Submitted By

Aryan Chudasama

230283111005

In partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

L.D. College of Engineering

Electronics and Communication Engineering

GUJARAT TECHNOLOGICAL UNIVERSITY, AHMEDABAD

University Area, Ahmedabad
Affiliated

Prof. Ketan K. Lad
Faculty guide

Prof. (Dr.) C. H. Vithalani
Head of the Department

Academic Year (2025 -2026)

L. D. College of Engineering,

Electronics and Communication Engineering

CERTIFICATE

Date: 10/07/2025

This is to certify that the project entitled "PCB Fault Detection: AI-Driven Optical PCB Inspection" has been successfully carried out by

Sr.	Name of Student	Enroll. No.
1.	Aryan Chudasama	230283111005

under my guidance in fulfilment of the Degree of Bachelor of Engineering in Electronics and Communication Engineering – 7th Semester of Gujarat Technological University, Ahmedabad during the academic year 2025-2026.

Internal Guide

Prof. Ketan K Lad

Electronics and Communication Engineering

Head of Department

Prof. (Dr.) C. H. Vithalani

Electronics and Communication Engineering

COURSE COMPLETION CERTIFICATE

CANDIDATE'S DECLARATION

I have finished my project report entitled "PCB Fault Detection: AI-Driven Optical PCB Inspection" and submitted to our respective guide. I have done my work proficiently with utter preciseness and to the best of my knowledge. It is a bona fide record of original project work carried out by me as a part of the Intel AI for Manufacturing Certificate Course under the supervision of Prof. Ketan K Lad and that no part of this report has been directly copied from any students' reports or taken from any other source, without providing due reference.

First Candidate's Name	:	**Aryan Chudasama**
Branch	:	**Electronics and Communication Engineering**
Enroll No.	:	**230283111005**
Signature	:	

Submitted to:

Prof. Ketan K Lad
L.D College of Engineering, Ahmedabad

ACKNOWLEDGEMENT

We would like to thank with respect all those who have provided us immense help and guidance during our project. We would like to thank our faculty guide **Prof. Ketan K Lad** for providing vision about the system and for giving us and opportunity to undertake such a great challenging and innovative work. We would also like to thank our external teachers from the Intel AI for Manufacturing Certificate Course: **Mr. Deep Patel** and **Mr. Kaiwalya Patil**, who have taught us much about artificial intelligence and how to apply it to real-world industrial applications We are grateful for the guidance, encouragement, understanding, and insightful support given in the development process. We would like to extend our gratitude to **Prof. (Dr.) C. H. Vithalani,** Head of the EC Department, LD College of Engineering, Ahmedabad, for his continuous encouragement and motivation.

Last but not the least, we would like to mention here that we are greatly indebted to each and everybody who has been associated with our project at any stage but whose name does not find a place in this acknowledgement.

Yours sincerely,

Aryan Chudasama (230283111005)

ABSTRACT

This report details the development of an AI-driven optical Printed Circuit Board (PCB) inspection system. The project's primary objective is to create a computer vision-based tool capable of significantly reducing manual inspection time and minimizing costly field returns in high-volume Electronics Manufacturing Services (EMS) environments.

The system employs a dual-model approach, utilizing two distinct YOLOv11 nano models:

- **Copper Track Defect Detection Model**: Designed to detect defects specifically within the PCB's copper tracks.
- **Component Detection Model**: Used to identify components soldered onto the PCB track and subsequently detecting missing components through comparison with a known good benchmark.

The solution integrates a responsive Flutter-based graphical user interface (GUI) with a high-performance Rust backend, leveraging ONNX Runtime for efficient machine learning inference.

This report covers the project's lifecycle, including problem scoping, data acquisition and extensive preprocessing, model training and evaluation, and deployment considerations, highlighting the technologies used and the measurable successes achieved in enhancing PCB quality control.

LIST OF FIGURES

LIST OF TABLES

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

- **AI:** Artificial Intelligence
- **API:** Application Programming Interface
- **CPU:** Central Processing Unit
- **CSV:** Comma-Separated Values
- **DLL:** Dynamic Link Library
- **EMS:** Electronics Manufacturing Services
- **FFI:** Foreign Function Interface
- **GPU:** Graphics Processing Unit
- **GUI:** Graphical User Interface
- **IoU:** Intersection over Union
- **JSON:** JavaScript Object Notation
- **mAP:** Mean Average Precision
- **ML:** Machine Learning
- **ONNX:** Open Neural Network Exchange
- **PCB:** Printed Circuit Board
- **PDF:** Portable Document Format
- **PR Curve:** Precision-Recall Curve
- **ROI:** Return on Investment
- **SAHI:** Slicing Aided Hyper Inference (Ref. [02])
- **SMD:** Surface-Mount Device
- **YOLO:** You Only Look Once (Ref. [01])

TABLE OF CONTENTS

CHAPTER 1: PROJECT OVERVIEW AND MANAGEMENT

1.1 Project Overview

This project focuses on developing an advanced AI-driven optical inspection system for Printed Circuit Boards (PCBs). The system aims to automate and enhance the quality control process in electronics manufacturing, specifically addressing the challenges of manual inspection.

The rapid advancements in manufacturing, particularly in the electronics industry, have led to increasingly complex Printed Circuit Boards (PCBs). These intricate designs necessitate meticulous quality control, traditionally performed through manual visual inspection.

However, manual inspection is inherently prone to human error, fatigue, and inconsistency, leading to potential defects passing through the production line. Such oversights can result in significant financial losses due to product recalls, rework, and damaged brand reputation.

The rise of Industry 4.0 and smart manufacturing paradigms emphasizes the integration of Artificial Intelligence (AI) to automate and enhance these critical processes, driving towards higher precision, efficiency, and cost-effectiveness.

1.1.1 Project Title

PCB Fault Detection: AI-Driven Optical Printed Circuit Board (PCB) Inspection System

1.1.2 Project Description

The core of this project is the development of a computer vision-based inspection tool designed to efficiently detect defects in PCBs directly from their images. This system is crucial for significantly reducing the time required for inspection and minimizing costly field returns, a capability particularly vital in high-volume Electronics Manufacturing Services (EMS) environments. The system is built to assist in quality control and inspection by identifying issues in both copper tracks and soldered components. It comprises a robust machine learning model training pipeline and a user-friendly desktop application for practical deployment.

1.1.3 Timeline

The project followed an iterative development process encompassing data acquisition, preprocessing, model training, evaluation, and deployment. The project work started at around 20 June 2025 and completed at 06 July 2025, well before the deadline of 15 June 2025.

1.1.4 Benefits and Return on Investment (ROI)

The implementation of this AI-driven PCB inspection system delivers substantial benefits across various facets of manufacturing operations, culminating in a significant return on investment:

- **Reduced Labor Costs**: By automating the visually intensive and repetitive task of PCB inspection, the system directly reduces the need for extensive manual labor. This translates to a reduction in labor costs associated with quality control, allowing human resources to be reallocated to more complex and value-adding tasks.

- **Decreased Rework and Scrap**: Early and precise detection of defects prevents faulty PCBs from progressing further in the assembly line. This proactive identification is projected to result in a decrease in rework expenses and a reduction in material scrap, as defects are caught before significant value is added to a flawed product.
- **Improved Product Quality and Reliability**: The AI model's consistent and objective detection capabilities eliminate human variability and fatigue, leading to a higher standard of quality control. This ensures that only defect-free PCBs proceed, enhancing the overall reliability and performance of final electronic products, and significantly reducing the likelihood of field failures.
- **Increased Throughput and Production Efficiency**: The automated system can inspect PCBs at a significantly faster rate than human operators. This leads to a considerable increase in production line throughput and overall operational efficiency.
- **Enhanced Data Insights and Traceability**: The system provides detailed data on detected defects, enabling manufacturers to identify recurring issues, analyze root causes, and implement targeted process improvements. This data-driven approach supports continuous quality enhancement and provides comprehensive traceability for auditing and compliance purposes.
- **Faster Time-to-Market**: By accelerating the inspection process and minimizing defects, the system contributes to a faster and smoother production cycle, ultimately leading to quicker product delivery and improved responsiveness to market demands.

1.1.5 Risks & Challenges

Challenges and risks identified and addressed during the project included:

- **Data Heterogeneity:** Inconsistent class naming and varied annotation formats across different datasets required extensive preprocessing.
- **Small Object Detection:** The presence of very tiny SMD components necessitated specialized techniques like tiling for training and Slicing Aided Hyper Inference (SAHI) for robust detection.
- **Deployment Constraints:** Balancing model performance with application size and execution provider compatibility (e.g., avoiding large CUDA binaries for broader device compatibility).

1.2 Project Objectives

The objectives of this project were clearly defined to ensure the development of an effective and practical PCB inspection solution.

1.2.1 Primary Objective

To develop a computer vision-based inspection tool that significantly reduces manual PCB inspection time and minimizes costly field returns in high-volume electronics manufacturing environments.

1.2.2 Secondary Objectives

- To accurately detect various types of defects specifically within the PCB's copper tracks.
- To accurately identify components soldered onto the PCB and detect missing components by comparing with a known good benchmark.
- To provide an intuitive and responsive graphical user interface (GUI) for users to easily upload images, select detection models, and visualize identified defects.
- To leverage high-performance backend technologies (Rust) for efficient and fast machine learning inference.
- To implement advanced image processing techniques (like SAHI) to improve detection accuracy for small objects.

1.2.3 Measurable Goals

The success of the project was assessed against the following measurable goals:

- **Copper Track Defect Detection Model Performance:** Achieve robust performance, as indicated by evaluation metrics such as Precision, Recall, F1-score, and Mean Average Precision (mAP) across various IoU thresholds. The optimal probability threshold for this model was targeted at approximately 25%.
- **Component Detection Model Performance:** Demonstrate strong performance using the same set of evaluation metrics, with an optimal probability threshold for deployment set at approximately 30%.
- **Application Functionality:** Successful deployment of a desktop application capable of processing PCB images, applying both defect detection models, and presenting clear visual reports of identified issues.

1.3 Project Planning

1.3.1 Development Approach and Justification

The project followed an iterative development approach, aligning with principles of the AI Project Lifecycle.

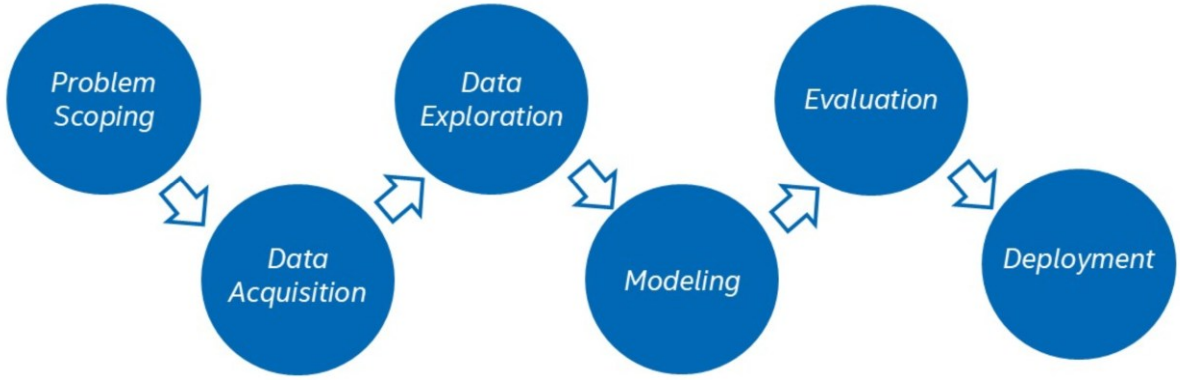


Figure 1.3.1: AI Project Lifecycle

This involved six distinct stages:

1. Problem Scoping
2. Data Acquisition
3. Data Preprocessing & Exploration
4. Modelling
5. Evaluation
6. Model Deployment

This structured approach allowed for continuous refinement and adaptation based on insights gained at each stage, which is particularly beneficial for machine learning projects where data characteristics and model performance can evolve.

The dual-model approach was justified by the mutual exclusivity of copper track defects and component presence, allowing for specialized and optimized models for each task.

The specifics of this AI Project Lifecycle followed are detailed in Section 3.1 (AI Project Lifecycle and Methodology).

1.3.2 Effort and Time Estimation

Given the short timeframe (approximately 16 days) and the complexity of the project, it implies intense, focused effort from the team.

- **Total Project Duration:** Approximately 16 days (from 20/06/2025 to 06/07/2025).
- **Total Human-Days:** 2 members * 16 days = 32 man-days.

1.3.3 Cost Estimation

The project had a budget constraint of 0 (no cost) beyond basic necessities.

- **Personnel Costs:** Free (academic project).
- **Infrastructure Costs:** Dell Inspiron i7 laptop, Nvidia 1050 TI GPU.

- **Software Costs:** Free and open-source (Flutter, Rust, Python libraries like Ultralytics, Keras, TensorFlow, PyTorch, Scikit-Learn).
- **Data Acquisition Costs:** Free (all datasets were freely available).
- **Total Estimated Cost:** Approximately ₹0, excluding electricity and internet.

1.3.4 Resource Availability

All resources were free and open-source, aligning with the requirements.

- **Development Tools:** Python, Rust, Flutter, Ultralytics, various ML and image processing libraries (Keras, TensorFlow, PyTorch, Scikit-Learn, `ort`, `image`, `ndarray`, `rayon`, Flutter `mobx`).
- **Operating System:** Likely Linux or Windows, given the tools used.
- **Version Control:** Git/GitHub (as repositories are hosted there).
- **Documentation:** Markdown, Jupyter Notebooks.

CHAPTER 2: PROBLEM SCOPING & SYSTEM ANALYSIS

2.1 Study of Current System

The traditional method for Printed Circuit Board (PCB) inspection largely relies on manual visual inspection. This involves human operators meticulously examining PCBs for various defects, including flaws in copper tracks and missing or incorrectly soldered components.

2.2 Problem and Weaknesses of Current System

Manual PCB inspections suffer from several critical weaknesses:

- **Time-Consuming:** The process is highly labor-intensive and slow, especially in high-volume manufacturing environments, leading to bottlenecks in the production line.
- **Error-Prone:** Human fatigue, inconsistencies in judgment, and the sheer complexity of modern PCBs can lead to missed defects, resulting in lower quality control.
- **Costly Field Returns:** Undetected defects that make it to the field can lead to product failures, customer dissatisfaction, and expensive recalls or repairs, significantly impacting revenue and brand reputation.
- **Subjectivity:** The quality of inspection can vary between different operators, leading to inconsistent defect identification.

2.3 Requirements of New System

Based on the weaknesses of the current system, the new AI-driven PCB inspection system was designed to meet the following key requirements:

- **Automation:** Automate the defect detection process to reduce reliance on manual labor.
- **Speed:** Significantly accelerate the inspection throughput.
- **Accuracy:** Improve the precision and recall of defect identification for both copper tracks and components.
- **Reliability:** Provide consistent and objective defect detection results.
- **Comprehensiveness:** Detect a wide range of defects, including various copper track anomalies and missing components.
- **User-Friendliness:** Offer an intuitive interface for easy operation by quality control personnel.

2.4 System Feasibility

The development and successful deployment of the PCB Fault Detection system demonstrate its feasibility across several dimensions:

- **Organizational Objectives:** The system directly contributes to the overall objectives of enhancing quality control, reducing operational costs, and improving product reliability in electronics manufacturing.
- **Technological Implementation:** The system has been successfully implemented using modern technologies (YOLOv11, Flutter, Rust, ONNX Runtime), proving that it can be built with current technological capabilities.
- **Integration:** While designed as a standalone desktop application, its modular architecture (separate UI and model training repositories, ONNX model export) suggests potential for future integration with existing manufacturing execution systems (MES) or quality management systems (QMS), although this was not explicitly part of the initial scope.

2.5 Activity & Process in New System

The proposed system operates through a streamlined process:

1. **Image Upload:** Users upload PCB images via the intuitive Flutter GUI.
2. **Model Selection:** Users select the appropriate detection model (Copper Track Defect Detection or Component Detection). The two models are mutually exclusive; if an image is selected as a track image, it cannot also be selected as a component image, as component presence can lead to false positives with the track model.
3. **Image Slicing (SAHI):** For both models, the uploaded images are sliced into smaller square sub-images. This process, known as Slicing Aided Hyper Inference (SAHI), is crucial for enabling the model to detect small components/track defects and improving detection accuracy for small objects by overcoming the fixed input resolution limitations of YOLOv11 (640x640).
4. **Inference:** The sliced sub-images are then sent to the high-performance Rust backend, where the selected YOLOv11 model performs inference using the ONNX Runtime.
5. **Result Stitching:** The detection results from the individual sub-images are then stitched back together to reconstruct the full image with detected defects or components.
6. **Defect Visualization & Reporting:** The GUI visualizes the identified defects (e.g., bounding boxes around track defects or indicators for missing components) and provides comprehensive reports, including a full report summarizing faulty PCBs. For component detection, the system first detects components on a known good benchmark PCB image and then compares subsequent PCB images against this benchmark to identify missing components.

2.6 Features of New System / Proposed System

The PCB Fault Detection system offers the following key features:

- **Intuitive Flutter GUI:** Provides a user-friendly interface for seamless interaction.
- **High-Performance Rust Backend:** Ensures fast and efficient machine learning inference.
- **Dual-Model Approach:**
  - **Copper Track Defect Detection Model:** Specialized for identifying defects within PCB copper tracks.
  - **Component Detection Model:** Designed to identify soldered components and detect missing ones by comparison.
- **YOLOv11 Integration:** Utilizes the state-of-the-art YOLOv11 object detection algorithm for both defect and component detection.
- **SAHI for Small Object Detection:** Incorporates Slicing Aided Hyper Inference to enhance detection accuracy for tiny components and track defects.
- **Comprehensive Reporting:** Generates visual reports, including initial screen views, missing components reports, track defects reports, and a full report summarizing faulty PCBs.

2.7 Main Modules of New System

The system is composed of several main modules and employs specific processes and techniques:

- `pcb-fault-detection` **Repository:** Hosts the core machine learning components, including data acquisition, preprocessing, model training, and evaluation code, along with trained model artifacts.
- `pcb_fault_detection_ui` **Repository:** Contains the Flutter desktop application that provides the user interface and integrates with the Rust backend for inference.
- **Copper Track Defect Detection Model:** A YOLOv11 Nano model trained specifically for identifying various copper track anomalies.
- **Component Detection Model:** A YOLOv11 Nano model trained for recognizing different electronic components.
- **Data Acquisition and Preprocessing Pipeline:** A series of scripts and Jupyter notebooks for gathering, cleaning, standardizing, and augmenting diverse PCB datasets.
- **Image Slicing (SAHI Implementation):** A Rust-based implementation of SAHI for dividing large images into smaller, overlapping tiles for improved small object detection during inference.
- **ONNX Model Export:** Trained models are exported to the ONNX format for cross-platform compatibility and efficient inference.
- **Rust Inference Engine:** Utilizes the `ort` (ONNX Runtime) Rust library for high-performance execution of the ONNX models.
- **Flutter GUI:** The front-end application built with Flutter for a responsive and intuitive user experience using reactive state management libraries like `mobx`.

2.8 Technology Stack Selection & Justification

The selection of the technology stack and methodologies was driven by the need for high performance, accuracy, and a responsive user experience:

- **AI Project Lifecycle (Methodology):** A structured approach was followed to ensure systematic development, from problem definition to deployment, allowing for iterative improvements and robust solution delivery.
- **YOLOv11 (Algorithm):** Chosen for its state-of-the-art performance in real-time object detection, balancing speed and accuracy, which is critical for industrial inspection.
- **Flutter (Software/Framework):** Selected for the GUI development due to its ability to build natively compiled applications for desktop from a single codebase, ensuring a responsive and intuitive user interface across different operating systems.
- **Rust (Programming Language/Backend):** Utilized for the high-performance backend due to its memory safety, concurrency features, and speed. It is ideal for computationally intensive tasks like machine learning inference and image processing.
- `flutter_rust_bridge` **(Framework):** Enables seamless communication and automatic FFI binding generation between the Flutter UI and the Rust backend, simplifying the integration of high-performance Rust code into the Flutter application.
- `ort` **(ONNX Runtime Rust Library):** Chosen for executing ML model inference in Rust. It leverages the ONNX format, providing a unified runtime for various ML frameworks and ensuring efficient deployment of the trained models.
- **ONNX (Model Format):** Selected as the model export format for its interoperability, allowing models trained in different frameworks (e.g., PyTorch via Ultralytics) to be deployed consistently across various platforms and runtimes.
- **DirectML (Execution Provider for ort):** Included as an execution provider for ONNX Runtime due to its broad compatibility with DirectX 12-capable hardware, offering GPU acceleration without the larger binary size associated with CUDA, making the application more accessible on a wider range of devices. CPU fallback is also included.
- `image` **and** `ndarray` **(Rust Libraries):** Used for efficient image manipulation and numerical operations within the Rust backend, particularly for image slicing and preprocessing tasks.
- `rayon` **(Rust Library):** Employed for easy and simple parallelism in Rust, allowing for efficient execution of parallel operations on data streams, which is beneficial for speeding up image processing and inference tasks.
- **Slicing Aided Hyper Inference (SAHI) (Technique):** Integrated into the inference pipeline to address the challenge of detecting small objects. By slicing images into overlapping sub-images, SAHI effectively increases the relative size of small objects within each sub-image, leading to improved detection accuracy.
- **Flutter MobX (State Management Library):** Chosen for its reactive state management capabilities, MobX provides an efficient and scalable way to handle application state. Its observable-based approach simplifies UI updates by automatically reacting to changes in data, ensuring a highly responsive and maintainable user interface, especially critical for complex real-time data visualization and interaction within the PCB inspection application.

CHAPTER 3: METHODOLOGY AND SYSTEM DESIGN

3.1 AI Project Lifecycle and Methodology

The project adhered to a standard AI Project Lifecycle methodology, which guided the development process from initial problem understanding to final deployment. This systematic approach ensured that each phase was thoroughly addressed, leading to a robust and effective solution.

The key phases included:

1. **Problem Scoping:** Defining the problem of manual PCB inspection and identifying the need for an AI-driven solution. This stage is explained in the Chapter 2 (Problem Scoping & System Analysis).
2. **Data Acquisition:** Gathering diverse datasets of PCB images with various defects and components. This stage is explained in Chapter 3 Section 3.3 (Data Acquisition).
3. **Data Preprocessing & Exploration:** Cleaning, standardizing, augmenting, and transforming raw data into a suitable format for model training. This stage is also explained in Chapter 3 Section 3.4 (Data Preprocessing & Exploration).
4. **Modeling:** Selecting and training appropriate machine learning models for the defined tasks. This stage is elaborated on in Section 5.3 (Model Training).
5. **Evaluation:** Rigorously assessing model performance using various metrics and techniques. This stage has an entire chapter dedicated to it: Chapter 6 (Evaluation and Results).
6. **Model Deployment:** Integrating the trained models into a functional application for real-world use. Included as a part of Chapter 6 in Section 6.4 (Model Deployment).

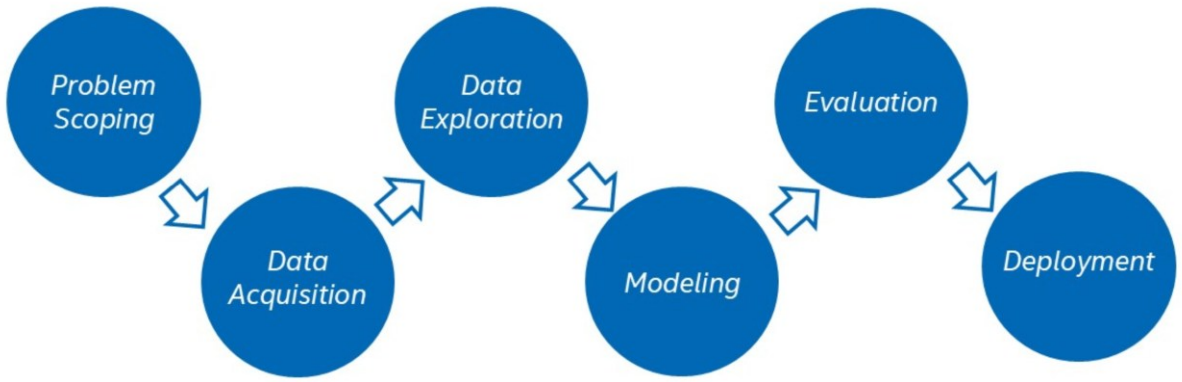


Figure 3.1.1: AI Project Lifecycle with Steps Explained

3.2 Dual-Model Approach Explanation

The system employs a dual-model approach, utilizing two distinct YOLOv11 models, each tailored for a specific purpose:

- **Copper Track Defect Detection Model:** This model is specifically designed to identify various types of defects that occur within the copper tracks of a PCB. It is trained on datasets annotated with anomalies such as shorts, spurs, open circuits, mouse bites, and foreign objects.
- **Component Detection Model:** This model is used to identify and localize electronic components soldered onto the PCB. Its primary application is in detecting missing components. This is achieved by first detecting components on an image of a known good benchmark PCB. Subsequently, images of other PCBs are selected and compared with this benchmark to identify any discrepancies, indicating missing components.

A crucial aspect of this dual-model approach is their mutual exclusivity. If an image is designated as a "track image" (meaning it's primarily for copper track inspection), it is not simultaneously processed by the component detection model.

This separation is necessary because a PCB image containing components can lead to numerous false positives when run through a model specifically trained for track defects, as components might be misidentified as defects.

Both models leverage slicing during inference to enhance detection accuracy for smaller features.

3.3 Data Acquisition

The project involved gathering various types of data from multiple sources to train both the Copper Track Defect Detection Model and the Component Detection Model.

3.3.1 For Copper Track Defect Detection Model:

These datasets, detailing defects on copper tracks, were used to train the CopperTrack model.

- DsPCBSD+
- MIXED PCB DEFECT DETECTION
- PCB_DATASET

3.3.2 For Component Detection Model:

These datasets contain annotated images of PCBs highlighting actual components. This data can be used to identify missing components by comparing an image with the output of a known good PCB.

- pcb-oriented-detection
- WACV pcb-component-detection
- FICS-PCB
  - Also at TRUST-HUB
  - And Roboflow
- PCB-Vision (Zenodo Download)
- CompDetect Dataset

3.3.3 For Demonstration (not model training):

This dataset contains annotated images of PCBs with missing soldered components. It was used in the final YouTube demo but not for model training.

- PCB defects.v2i.yolov11

3.4 Data Preprocessing & Exploration

Extensive data preprocessing and exploration were critical steps to ensure the quality and consistency of the datasets for effective model training. This involved handling inconsistent class naming, converting various annotation formats to YOLO format, and applying image enhancements.

3.4.1 For the Copper Track Defect Detection Model

- **DsPCBSD+ Dataset:** This dataset was the most comprehensive in terms of classes, and its class names were adopted as the standard for the entire aggregated dataset. Due to inconsistent class naming across datasets, the following mappings were applied to standardize labels.

```
DSCPBSD_MAP = {
    0: SHORT,
    1: SPUR,
    2: SPURIOUS_COPPER,
    3: OPEN,
    4: MOUSE_BITE,
    5: HOLE_BREAKOUT,
    6: SCRATCH,
    7: CONDUCTOR_FOREIGN_OBJECT,
    8: BASE_MATERIAL_FOREIGN_OBJECT,
}
```

- **MIXED PCB DEFECT DATASET:** Sourced from Mendeley Data, this dataset contained similar data but with different labeling, necessitating label mapping:

```
MIXED_PCB_DEFECT_DATASET_MAPPING = {
    0: MISSING_HOLE,
    1: MOUSE_BITE,
    2: OPEN,
    3: SHORT,
    4: SPUR,
    5: SPURIOUS_COPPER,
}
```

- **PCB_DATASET:** From Kaggle, this dataset was in Pascal VOC format and required conversion to YOLO format. The mapping applied was

```
PCB_DATASET_MAPPING = {
    "missing_hole": MISSING_HOLE,
    "mouse_bite": MOUSE_BITE,
    "spurious_copper": SPURIOUS_COPPER,
    "short": SHORT,
    "spur": SPUR,
    "open_circuit": OPEN,
}
```

This dataset contains full-sized images of complete PCB tracks, and is consequently designated as the **Large PCB Image Dataset**.

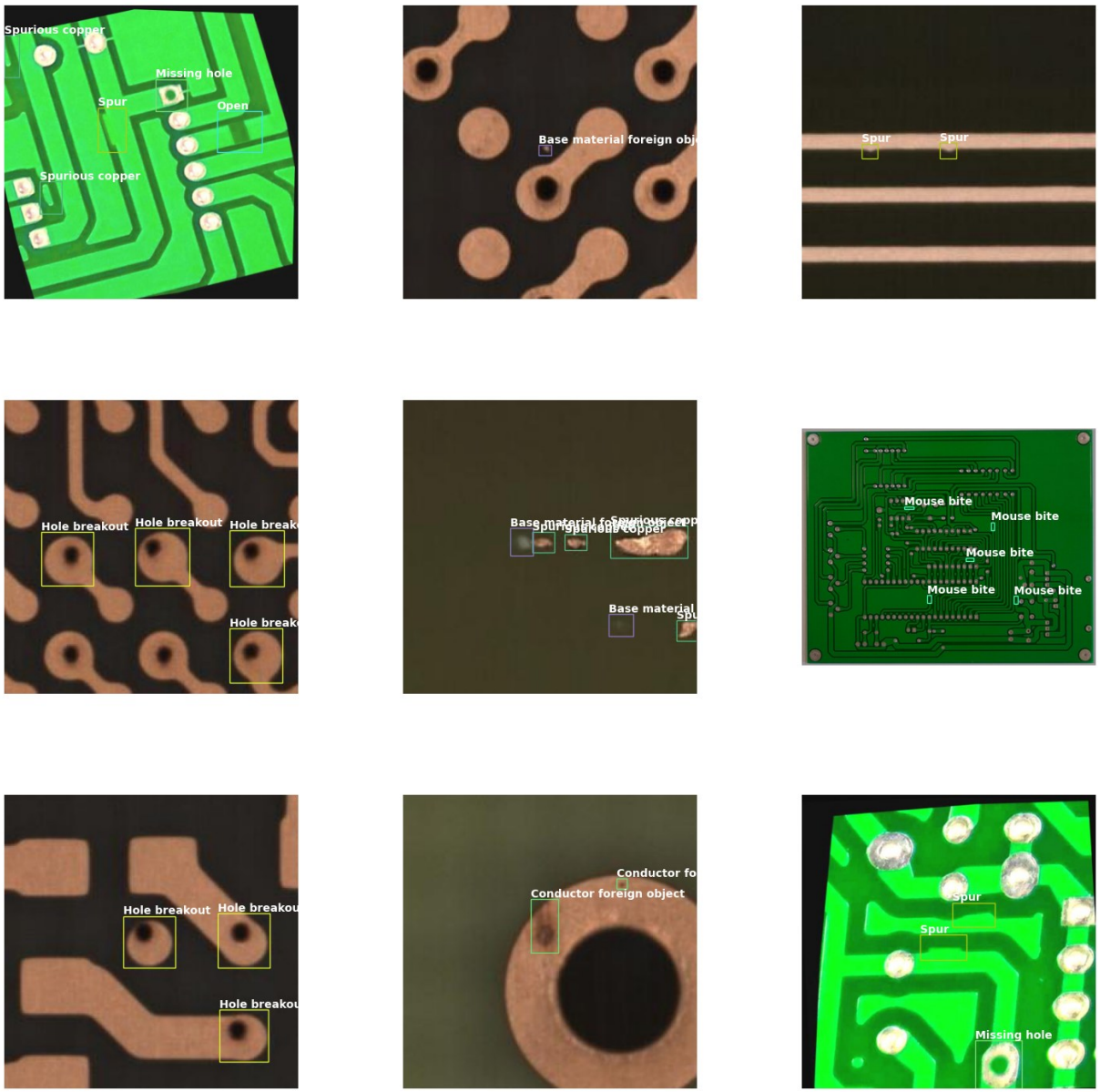


Figure 3.4.1 Copper Track Defects Final Dataset Sample Images

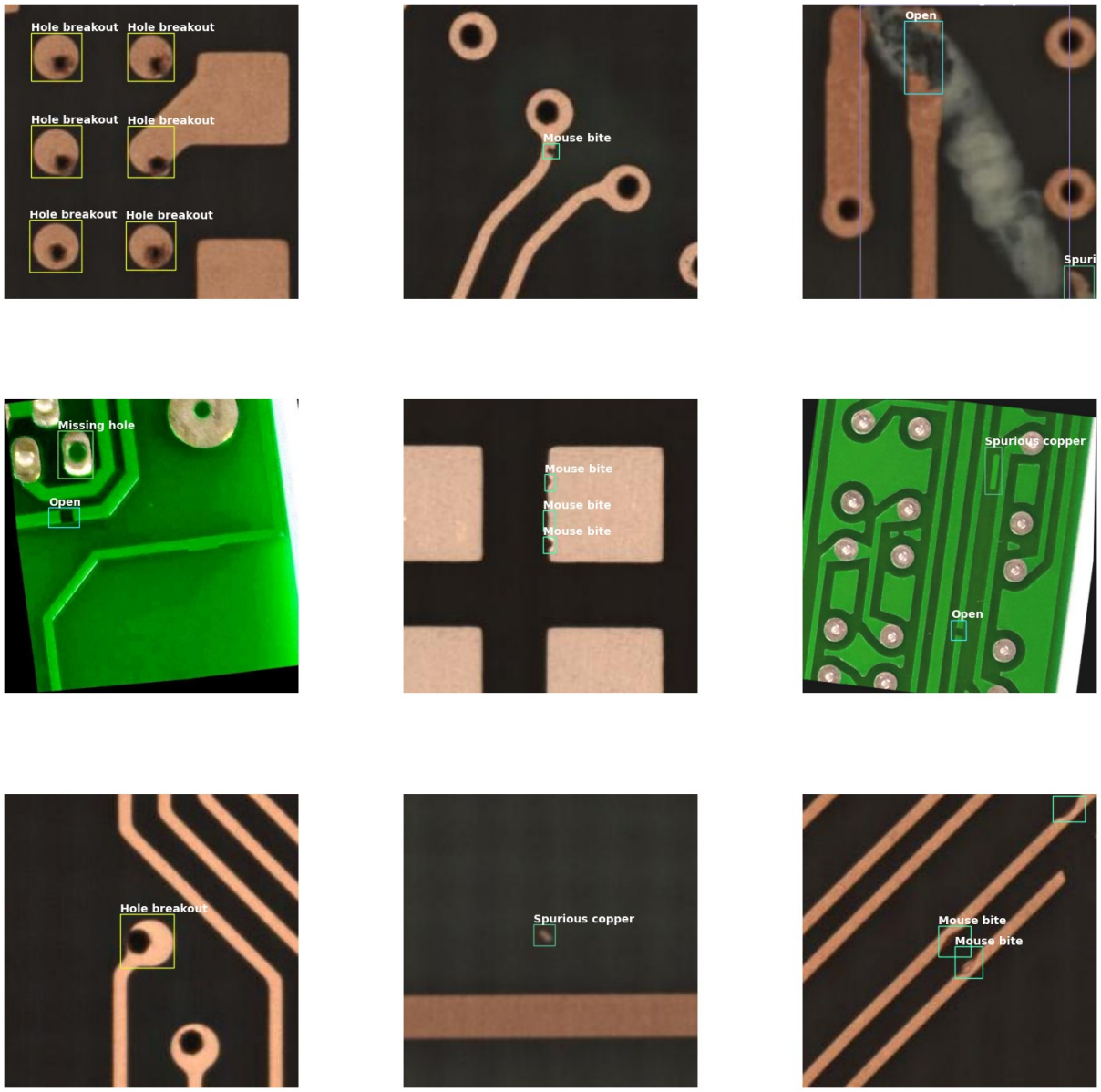


Figure 3.4.2 Copper Track Defects Final Dataset Sample Images 2

3.4.2 For the Component Detection Model

The class names for the components are as follows:

```
@enum.verify(enum.UNIQUE, enum.CONTINUOUS)
class Component(enum.IntEnum):
    battery = 0
    button = 1
    buzzer = 2
    capacitor = 3
    clock = 4
    connector = 5
    diode = 6
    display = 7
    fuse = 8
    heatsink = 9
    ic = 10
    inductor = 11
    led = 12
    pads = 13
    pins = 14
    potentiometer = 15
    relay = 16
    resistor = 17
    switch = 18
    transducer = 19
    transformer = 20
    transistor = 21
```

CompDetect Dataset:

From Roboflow Universe, this dataset was useful for component detection. However, direct download of original, higher-quality images was not possible, and it contained many augmented images.

The process involved forking the project on Roboflow, using a custom script `roboflow_download.py` to save images, and then another custom script `roboflow_save_labels.py` to save data in Roboflow JSON API response format. This JSON API format was then converted to the required YOLO format.

FICS-PCB:

This dataset, detailed in the paper FICS-PCB: A Multi-Modal Image Dataset and available for download from TRUST-HUB, presented challenges with its large size (~79GB) and complex annotation format (CSV for labels, JSON for bounding boxes).
A mirrored version on Roboflow Universe allowed for similar processing as the CompDetect dataset, so that is what was opted for. The Roboflow mirror also contained duplicate images from the WACV dataset which were removed.

PCB-Vision:

This dataset, described in the arXiv paper and downloadable from Zenodo (11GB), required significant preprocessing. A dedicated Jupyter Notebook, `preprocess-pcb-vision.ipynb`,

was used for conversion to a YOLO-compatible format. The original dataset specified classes using a grayscale image mask where pixel values indicated component presence:

- 0 = Nothing
- 1 = IC (represented as red)
- 2 = Capacitor (represented as green)
- 3 = Connectors (represented as blue)

Figure 3.4.3 PCB-Vision HSI Masks

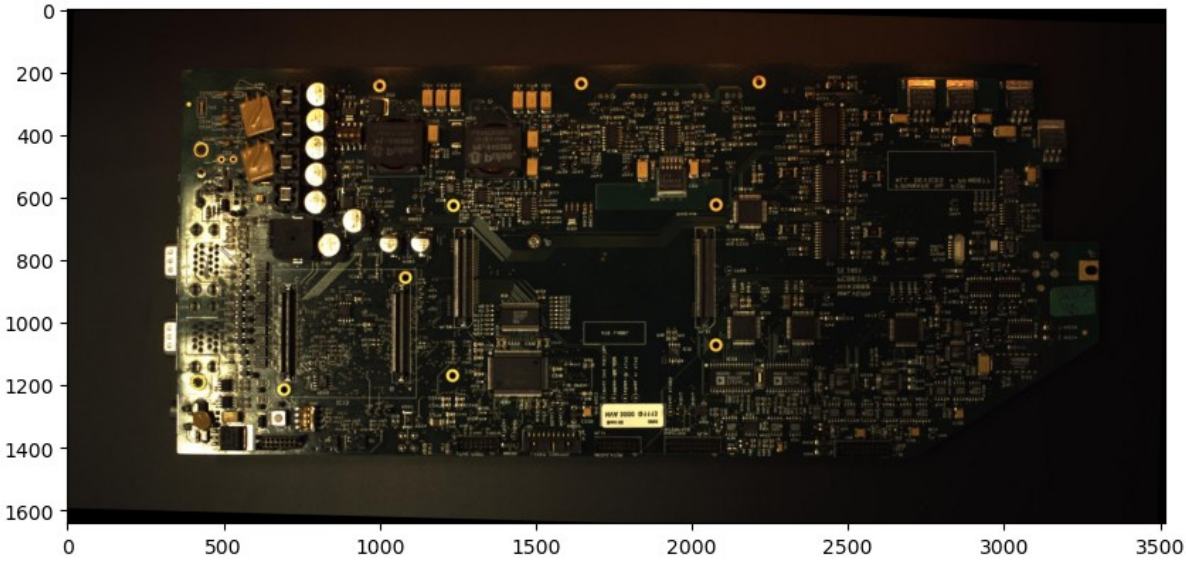


Figure 3.4.4 PCB-Vision: Sample Image

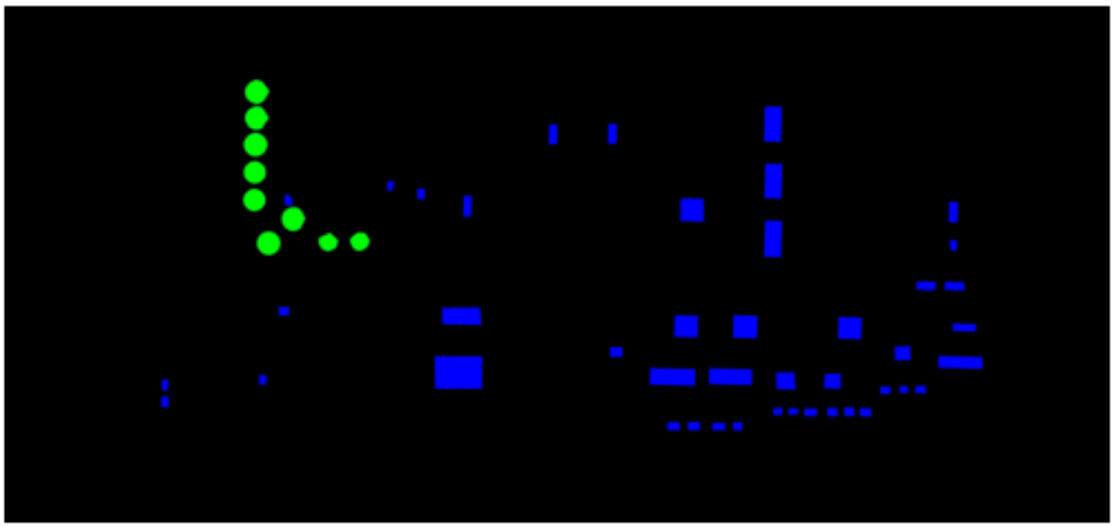


Figure 3.4.5 PCB-Vision: Mask Corresponding to the above Sample Image

The `preprocess-pcb-vision.ipynb` then performed the following steps:

- **Image Straightening:** Used PCB mask files to detect and correct tilted PCBs, preventing issues with YOLO's bounding box predictions. This involved finding the largest contour, determining the smallest rotated bounding box, and rotating the image and component mask accordingly.
- **Bounding Box Derivation:** From the component masks, separate component contours were identified, and bounding boxes fitting these contours were derived and saved as YOLO labels.
- **Color Correction:** Basic color correction was applied to often dark images by clipping color channel values to their 97.5th percentile.

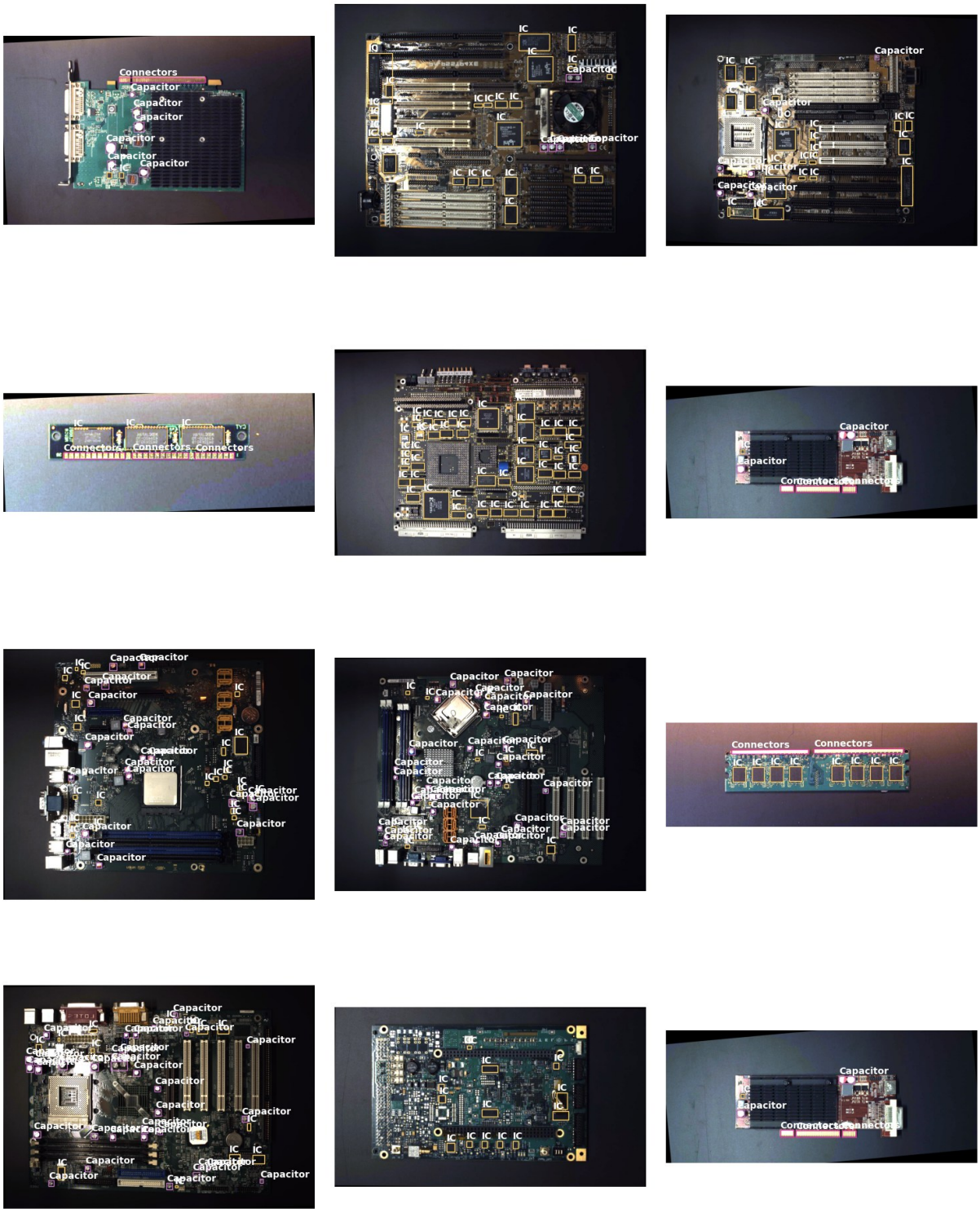


Fig 3.4.6 PCB-Vision Sample Images After Conversion to YOLO Format

WACV: pcb-component-detection

From Chia-Wen Kuo's Research Page, this Pascal VOC formatted dataset also required conversion to YOLO format.

pcb-oriented-detection

From Kaggle, this dataset contained oriented bounding boxes, which were converted to regular bounding boxes by finding the tightest fitting regular boxes. Similar color correction as for PCB-Vision was applied. This dataset also required a comprehensive class mapping due to numerous redundant classes.

3.4.3 Tiling for Training

Many datasets contained very tiny bounding boxes, especially for small SMD components. To improve model training by making these components comparatively larger, images were split into larger tiles, and their bounding box annotations were adjusted. This process is analogous to Slicing Aided Hyper Inference (SAHI) but is performed during training. The process works as follows:

- Reads an image and its bounding boxes.
- Identifies the smallest dimensions among all bounding boxes.
- Determines the necessary tile size to ensure the smallest bounding box occupies at least 3% of the tile's area.
- Crops the image into tiles with at least 10% overlap, redistributing the overlap to minimize clipping at edges.

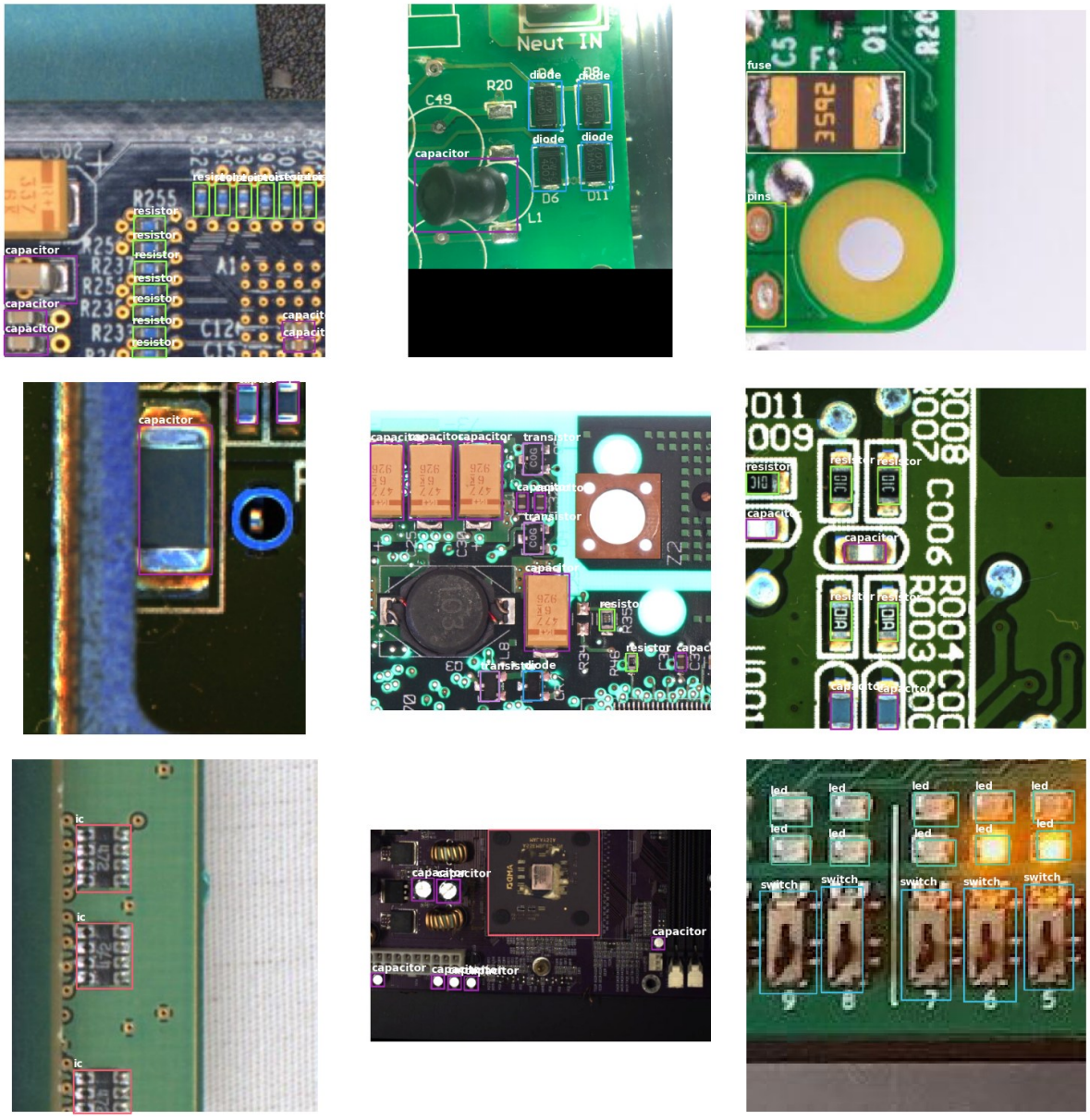


Fig 3.4.7 Component Detection Final Dataset Sample Images

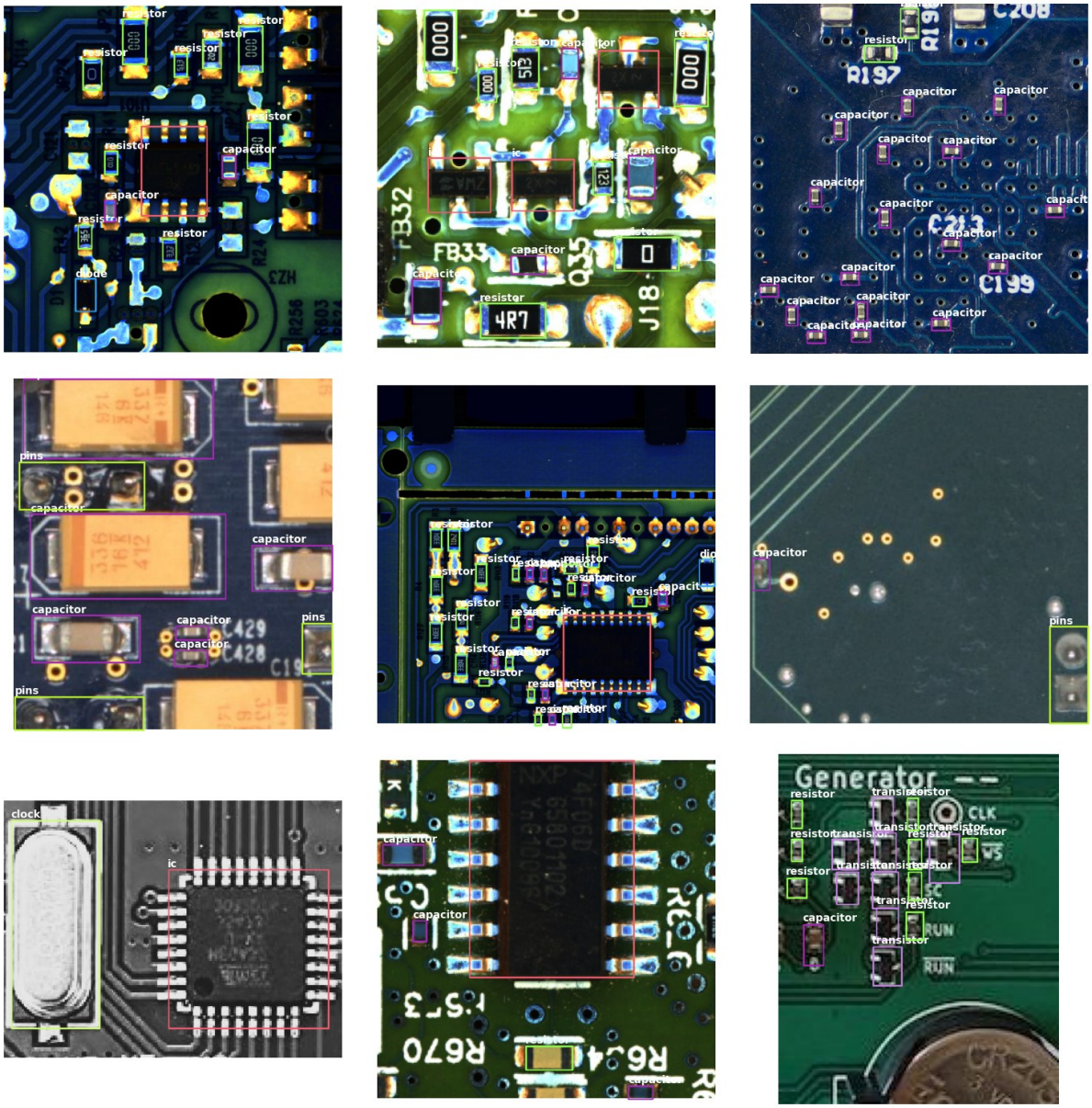
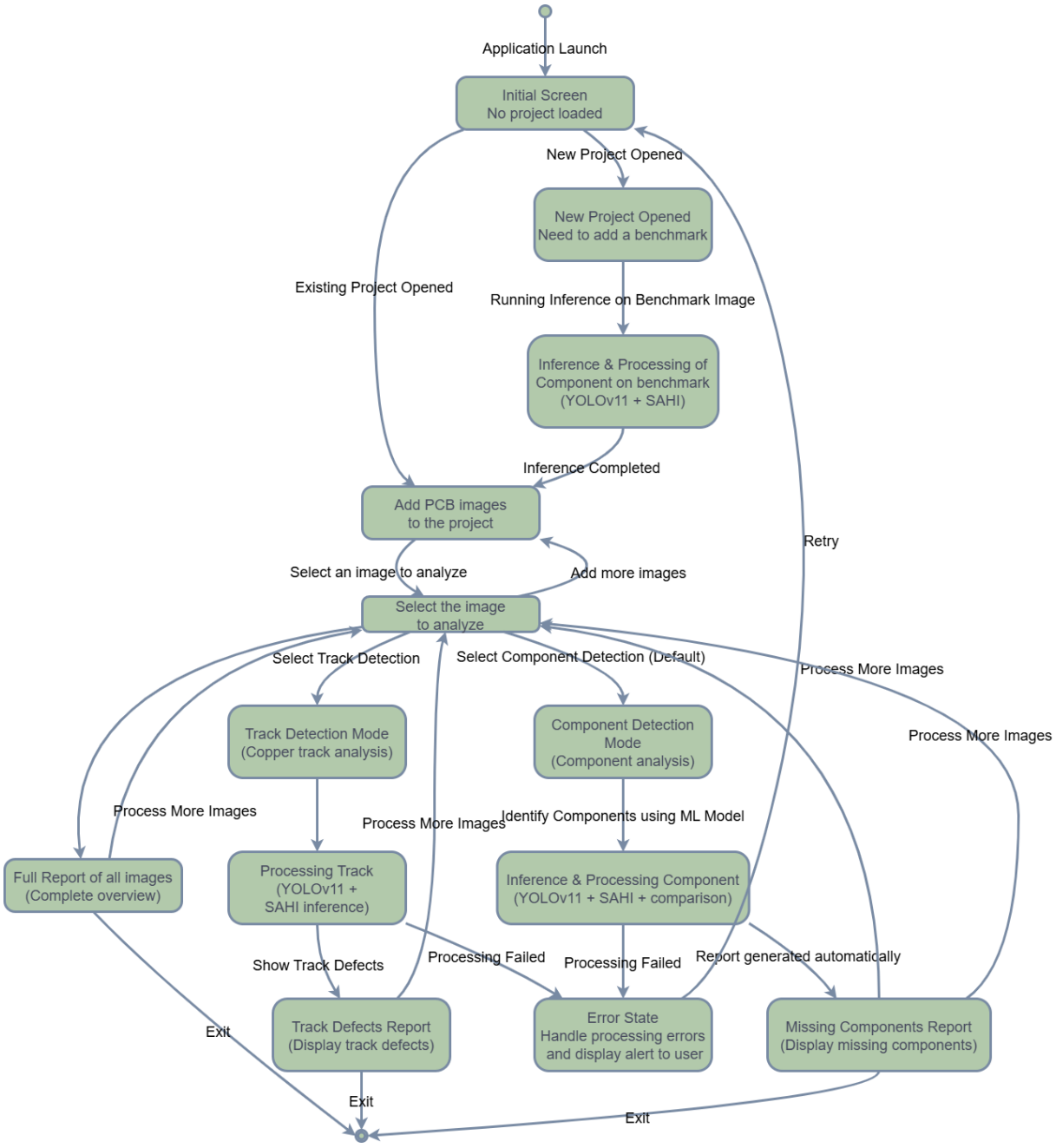


Fig 3.4.8 Component Detection Final Dataset Sample Images 2

3.5 Input / Output and Interface Design

The user interface for the PCB Fault Detection system is designed for intuitive interaction, allowing users to easily manage and visualize the inspection process.

3.5.1 State Transition Diagram



3.5.2 Samples of Forms, Reports and Interface

The Flutter GUI provides clear visual feedback and reports. Screenshots illustrate the user experience:

No project selected screen:

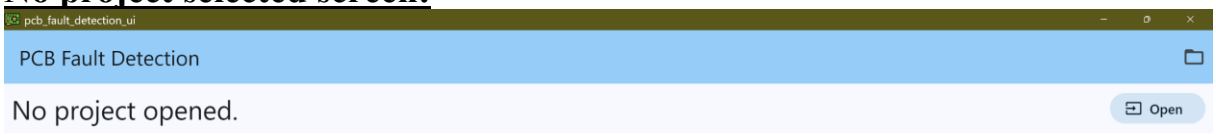


Figure 3.5.1 Initial Screen

Select a project by clicking on open & opening a folder:

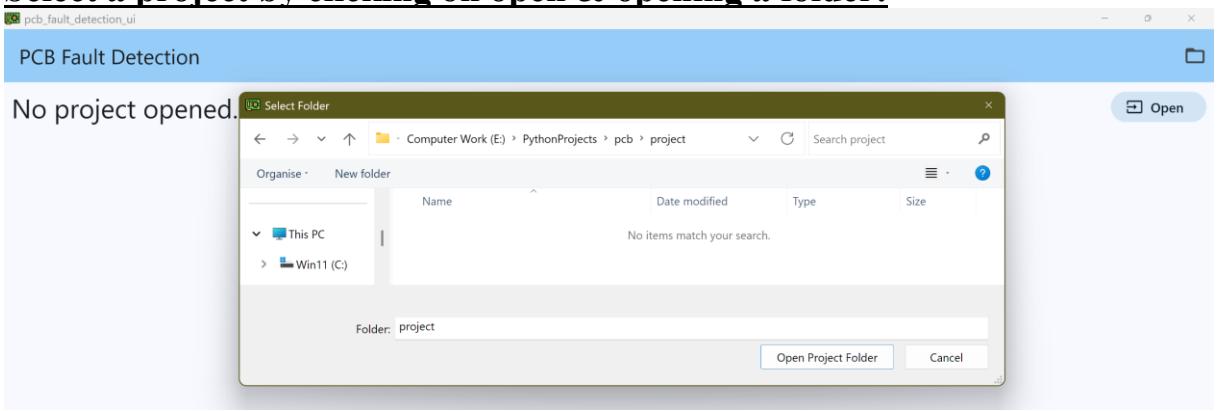


Figure 3.5.2 Project Selection

After project selection:

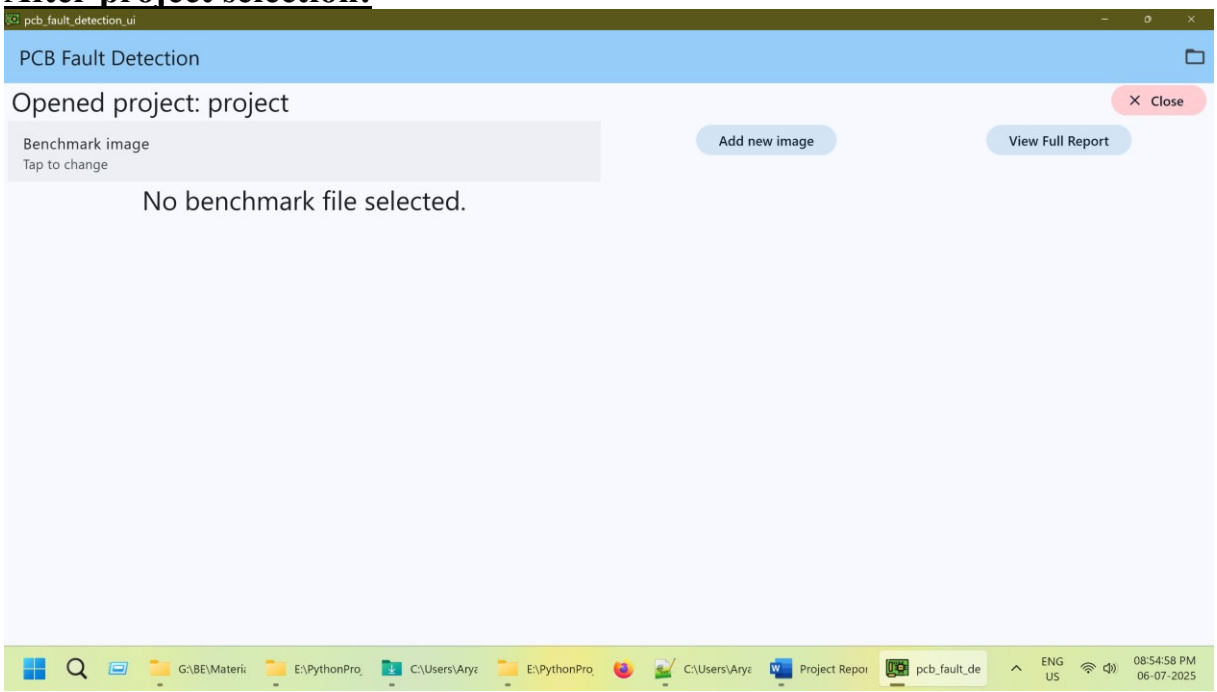


Figure 3.5.3 Blank Project Open

Select a benchmark image:

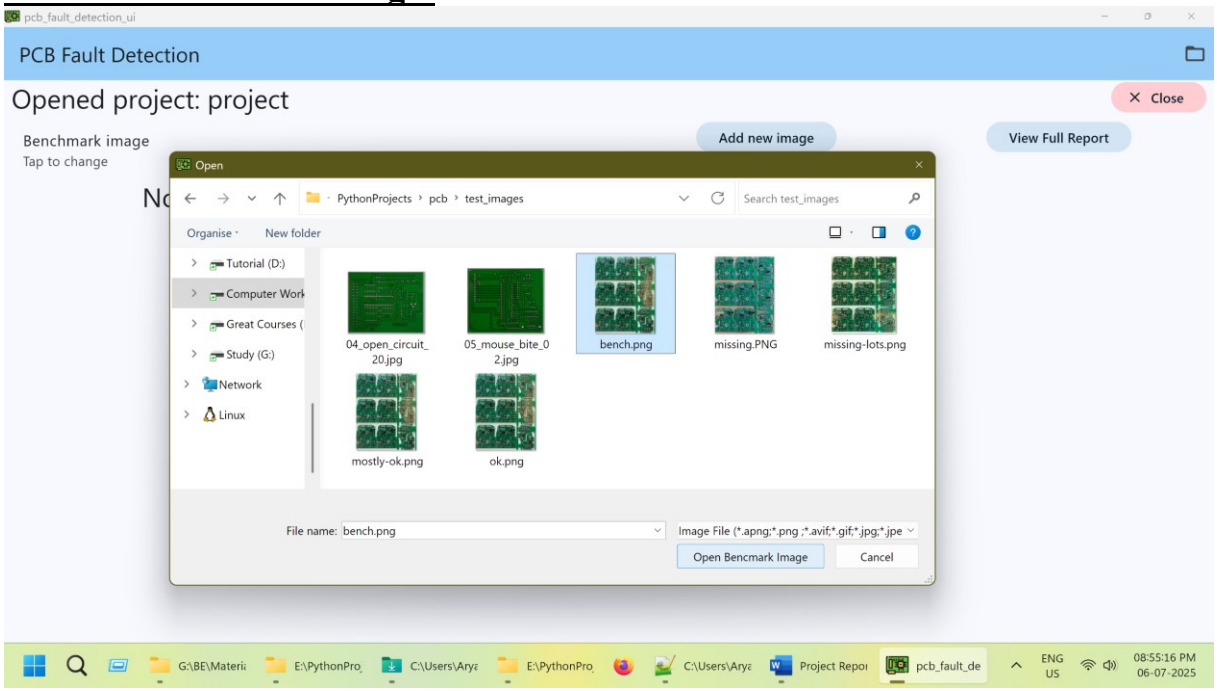


Figure 3.5.4 Select a benchmark image

After selecting click Run Inference and then wait:

Figure 3.5.5 Wait for inference to run on the benchmark image

View the components on the benchmark image:

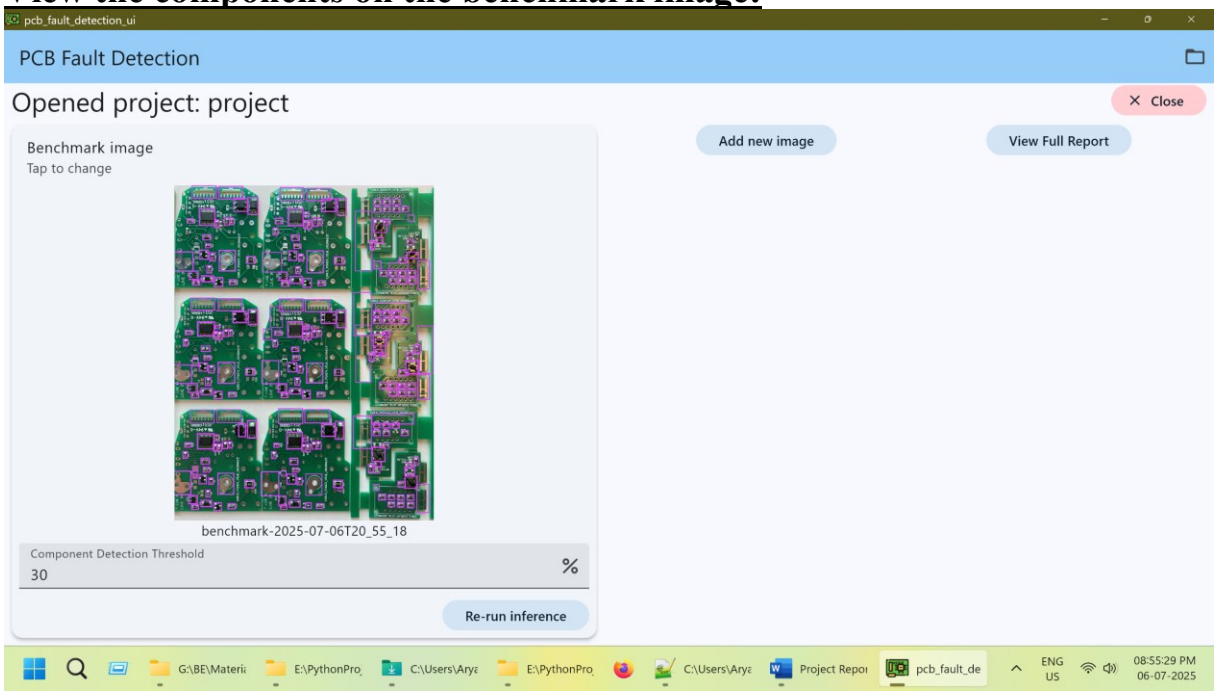


Figure 3.5.6 View benchmark components

Also note that you can zoom into any image view using in the entire application.

Add other images:

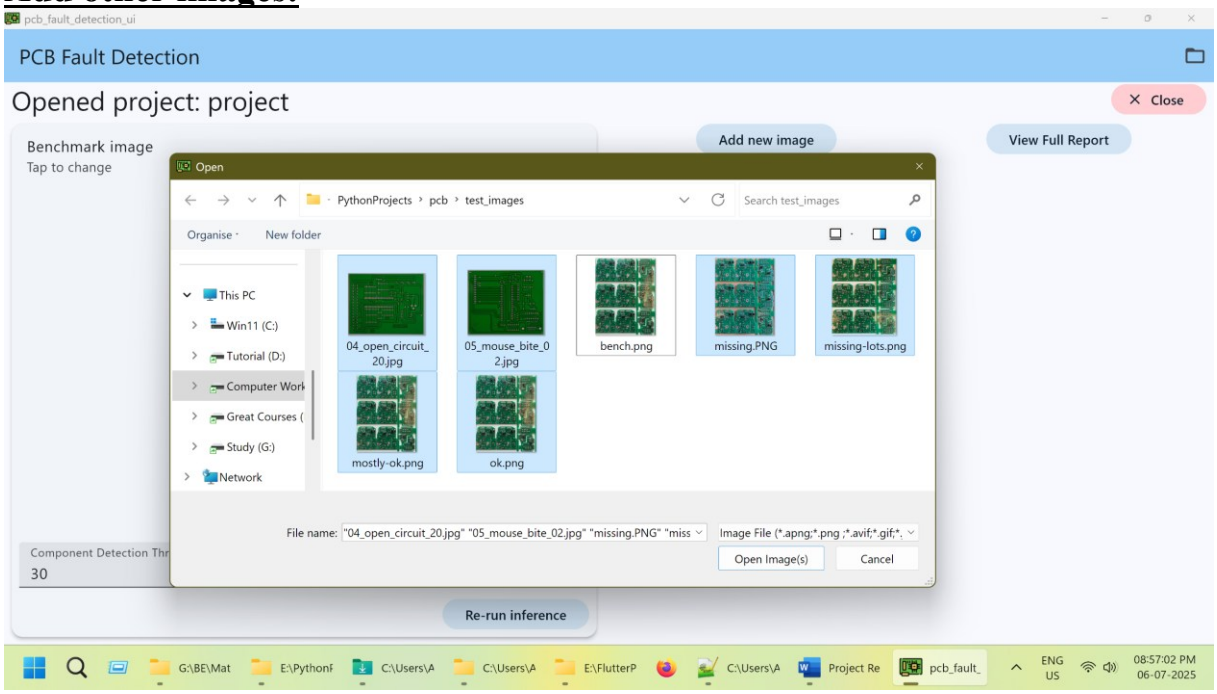


Figure 3.5.7 Adding other images

Observe that they are added:

Figure 3.5.8 Main Screen with images added

Open a PCB track image, and tick "Is this a PCB track image?" to mark it as a track image:

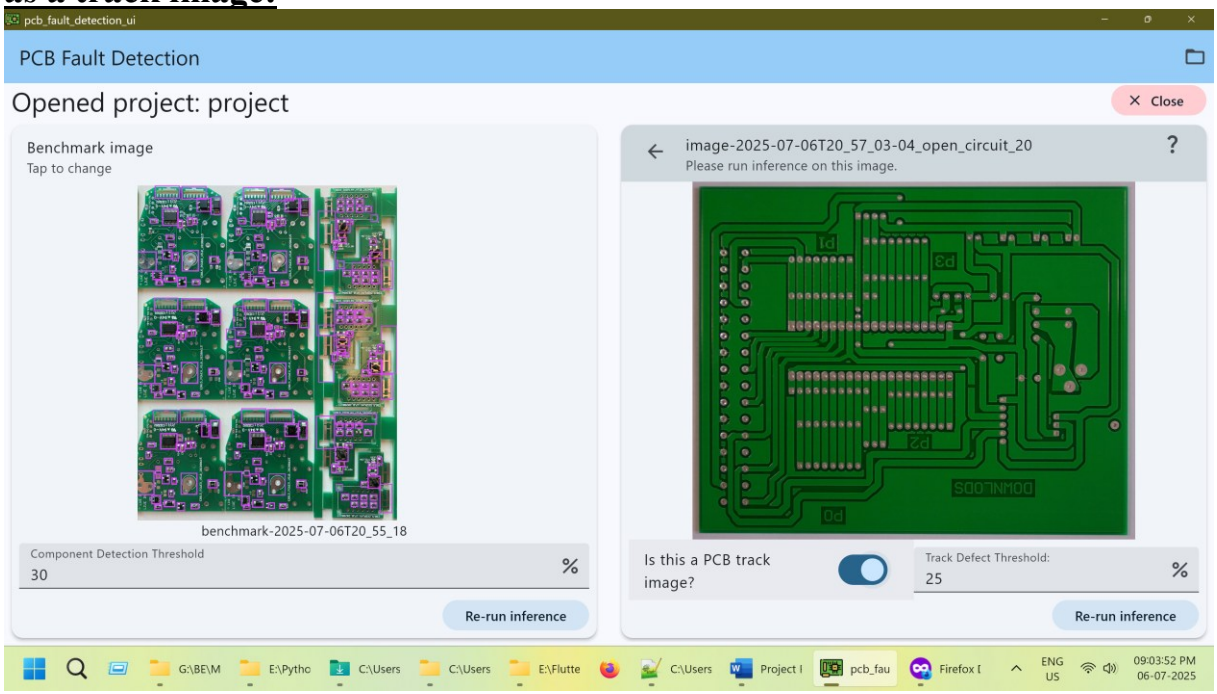


Figure 3.5.9 Track Defect Image Mode Selection

Click on Run Inference to view the PCB track defects:

This report is generated only when the user selects "Is this a PCB track image?" in the UI, and the runs inference on the image, displaying identified defects within the copper tracks:

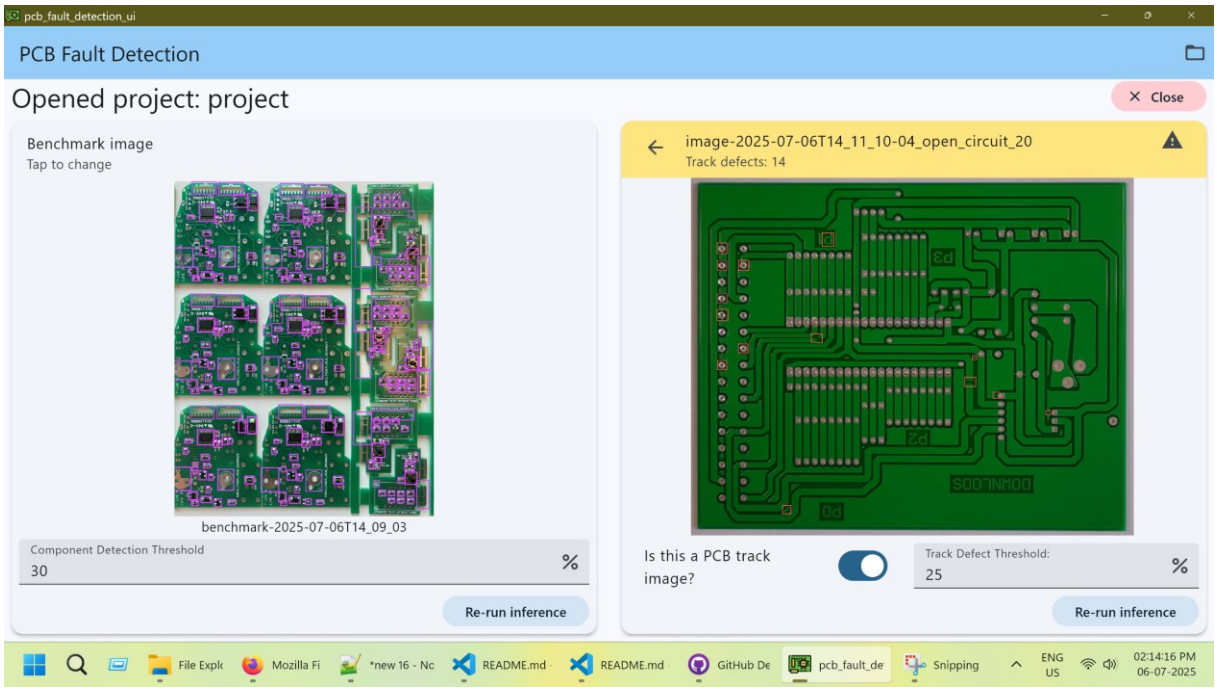


Figure 3.5.10 Track defects in red bounding boxes

Zoom in to see the defects better:

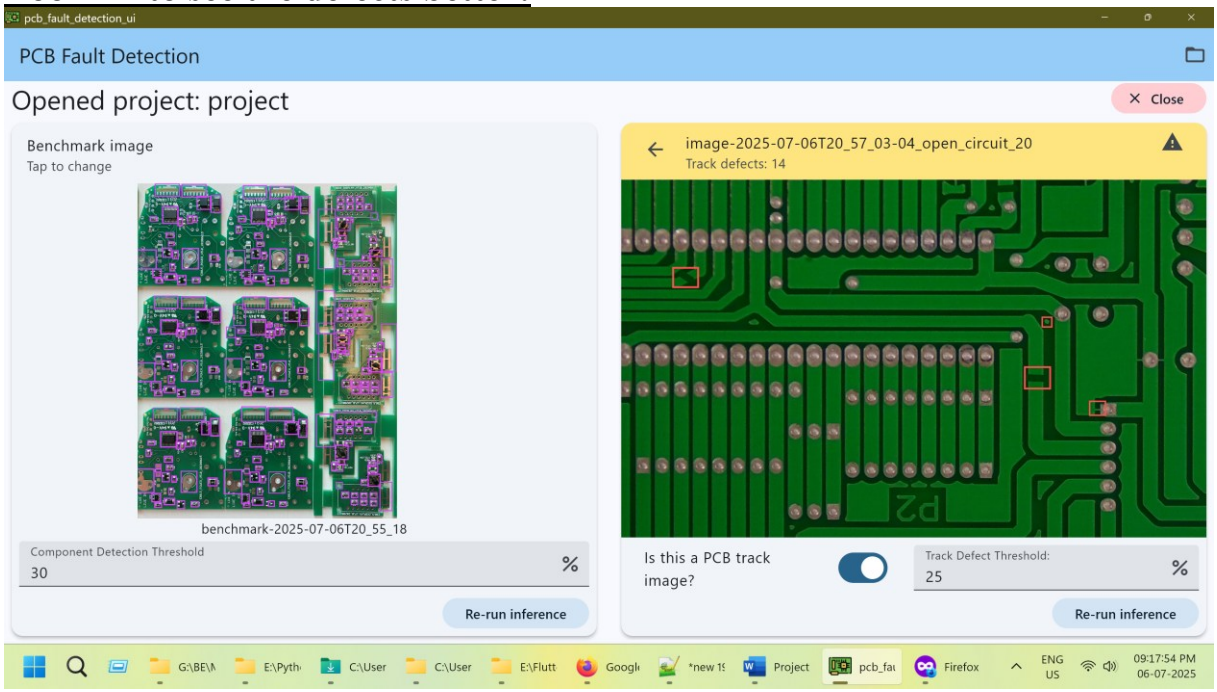


Figure 3.5.11 Zoomed in track defects

Go back to the main screen and select a PCB image with components on it:

Figure 3.5.12 PCB Components Mode

It is important to note that the purple bounding boxes represent components present in the benchmark image but absent from the current image, while the red delineations indicate components found in the current image that are not present in the benchmark. Consequently, upon initial image display, all benchmark components are highlighted in purple, as no inference has yet been performed on the current image.

Click on “Re-run inference” to run inference on the image.

Observe the missing components by selecting it from the dropdown:

This report visualizes detected missing components on the PCB, highlighting areas where components are expected but not found. Here, two ICs are missing as shown in the screenshot:

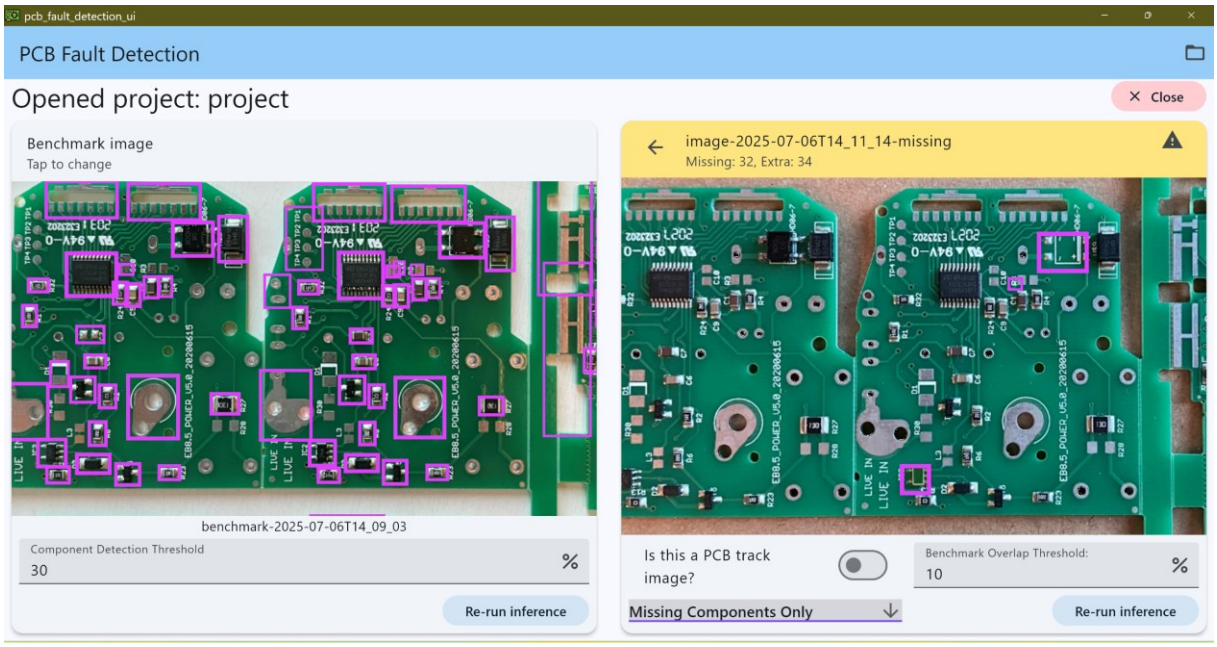


Figure 3.5.13 Missing Components View

Go back to the main screen and select “View Full Report”

The full report provides a concise overview, allowing users to see at a glance which PCB images are faulty:

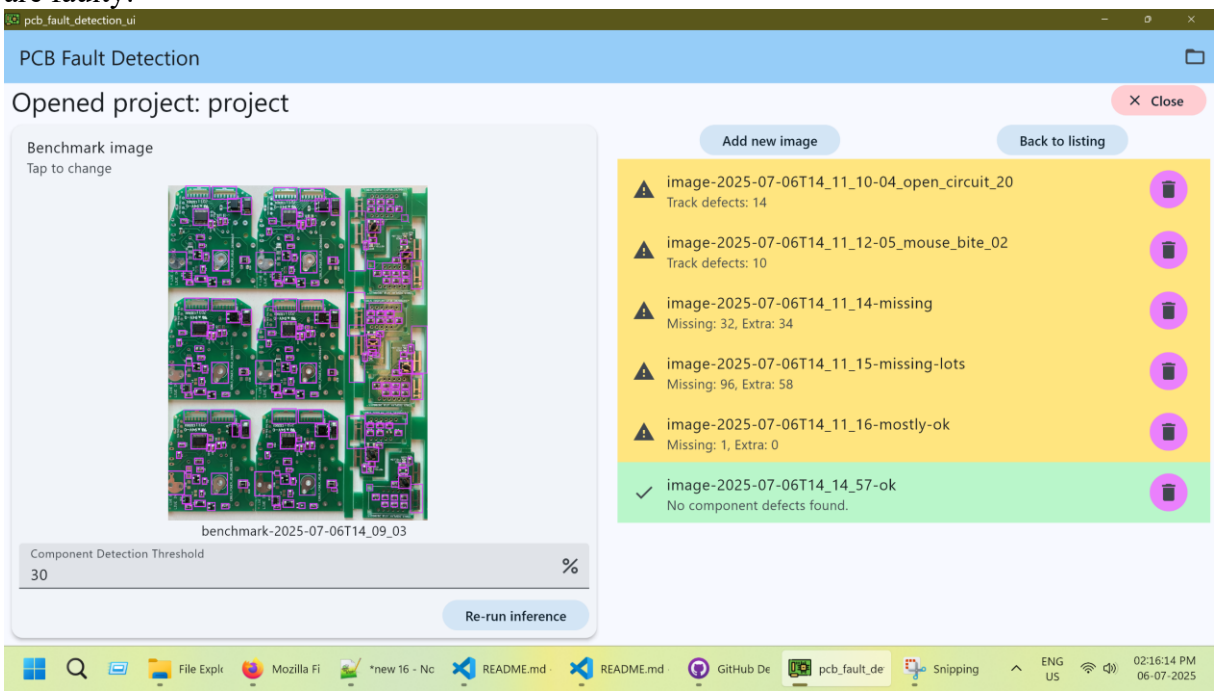


Figure 3.5.14 Full Report

Click on “Back to listing” to go back to the main page.

CHAPTER 4: TECHNOLOGIES USED

The PCB Fault Detection project leverages a modern and high-performance technology stack to achieve its objectives.

4.1 Programming Languages

- **Python:** Used extensively for the machine learning components, including data acquisition, preprocessing, model training (with Ultralytics), and evaluation.
- **Rust:** Employed for the high-performance backend of the desktop application, handling machine learning inference, advanced image processing, and parallelism.
- **Dart:** The primary language for developing the graphical user interface using the Flutter framework.

4.2 Development Frameworks and Libraries

- **Ultralytics (Python):** The framework used for training and evaluating the YOLOv11 models.
- **Flutter (Dart):** The UI toolkit used to build the cross-platform desktop application, providing a responsive and intuitive user experience.
- `flutter_rust_bridge` **(Rust/Dart):** A framework that facilitates communication between Flutter (Dart) and Rust, automatically generating the necessary Foreign Function Interface (FFI) bindings.
- `ort` **(Rust):** A Rust library that provides bindings to the ONNX Runtime, enabling efficient execution of ONNX models for machine learning inference.
- `image` **(Rust):** A Rust crate for image processing, used for tasks like image slicing and manipulation.
- `ndarray` **(Rust):** A Rust library for N-dimensional arrays, providing efficient numerical computation capabilities for image data.
- `rayon` **(Rust):** A data-parallelism library for Rust, used to easily and simply run parallel operations on streams of data, enhancing performance for image processing and inference.
- **Flutter MobX (Dart):** A robust and scalable reactive state management library for Flutter, MobX was chosen to manage the complex application state within the UI. By leveraging observables, actions, and reactions, it ensures that the user interface efficiently updates only when relevant data changes, leading to highly performant and maintainable code, which is crucial for a responsive PCB inspection application.

4.3 Development Tools

- **Jupyter Notebooks:** Used for interactive development, experimentation, and documentation of data preprocessing and model training steps.
- **Git/GitHub:** Version control system and platform for collaborative development and code hosting.
- **ONNX:** The Open Neural Network Exchange format, used for exporting trained models to ensure interoperability and efficient deployment across different runtimes.

4.4 Testing Tools

The primary testing and evaluation tools are inherent to the Ultralytics framework used for model training, which provides comprehensive metrics and visualization tools for assessing model performance like:

- **Object Detection Metrics**:
  - Precision
  - Recall
  - F1-score
  - Box-MAP
  - Box-MAP50
  - Box-MAP75
- **Visual Plots**:
  - Confusion Matrix
  - Normalized Confusion Matrix
  - Precision-vs-Confidence Curve
  - Recall-vs-Confidence Curve
  - F1-vs-Confidence Curve
  - Precision-vs-Recall Curve

All these topics are further elaborated on in Chapter 6 (Evaluation And Results).

4.5 Cloud Services

While the application itself is a desktop application, datasets were sourced from various online platforms, which can be considered cloud-based data repositories:

- Roboflow Universe
- Mendeley Data
- Kaggle
- Zenodo
- TRUST-HUB

Furthermore, the consolidated component detection dataset has been deposited on Kaggle, given the intricate aggregation and preprocessing pipeline associated with the dataset. This action facilitates access for other researchers and practitioners who may benefit from this aggregated data for their respective PCB component detection initiatives.

4.6 APIs and Web Services

- **Roboflow API:** Used during the data acquisition phase for programmatically downloading datasets from Roboflow Universe.

4.7 Why Rust?

Rust was chosen for the backend of the PCB Fault Detection UI due to its unique combination of performance, memory safety, and concurrency features, which are highly beneficial for a high-performance machine learning application:

- **High Performance:** Rust offers performance comparable to C++ while providing strong guarantees about memory safety, making it ideal for computationally intensive tasks like machine learning inference.
- **Memory Safety:** Rust's ownership system and borrow checker eliminate common programming errors like null pointer dereferences and data races at compile time, leading to more reliable and stable software. All of this can be summarized by the simple & succinct phrase "**Aliasing XOR Mutability**", which is at the heart of Rust's design philosophy & architecture.
- **Concurrency:** The `rayon` library in Rust provides a straightforward way to achieve data parallelism, allowing for efficient processing of image data and model inference across multiple CPU cores.
- **ONNX Runtime Integration:** The `ort` Rust library provides robust bindings to the ONNX Runtime, enabling the efficient execution of machine learning models in the ONNX format. This allows the application to leverage pre-trained models from various frameworks.
- **Image Processing Capabilities:** Libraries like `image` and `ndarray` in Rust offer powerful and efficient tools for image manipulation and numerical operations, which are essential for tasks such as image slicing (SAHI implementation).
- `flutter_rust_bridge`**:** This framework specifically allows Flutter (Dart) applications to seamlessly interface with Rust code, automatically generating the necessary Foreign Function Interface (FFI) bindings. This greatly simplifies the integration of the high-performance Rust backend with the responsive Flutter GUI.

The decision to compile `ort` with DirectML support (and CPU fallback) as the sole execution provider, rather than including CUDA binaries, was a pragmatic choice to manage application size. While CUDA offers superior performance on compatible GPUs, its binaries significantly increase the application's footprint. DirectML provides a good balance of GPU acceleration and broader device compatibility, making the application more accessible for demonstration and general use on a wider range of hardware. The modularity of `ort` allows for easy addition of other execution providers if specific hardware acceleration (e.g., CUDA) becomes a critical requirement in the future.

CHAPTER 5: IMPLEMENTATION

5.1 Implementation Platform / Environment

The project's implementation spans multiple environments for development and deployment:

- **Development Environment:**
  - **Python:** Used for all machine learning pipeline development, including data preprocessing scripts and model training notebooks (e.g., Jupyter Notebooks).
  - **Rust:** Used for developing the high-performance backend logic and image processing functionalities.
  - **Flutter (Dart):** Used for developing the cross-platform desktop application's user interface.
- **Deployment Environment:**
  - The final application is a desktop application, with binaries available for Windows, and potentially adaptable for Linux/macOS, as indicated by the compilation instructions.

5.2 Process, Program & Technology Module Specifications

The implementation involved distinct processes and modules:

- **Data Preprocessing Modules:** Python scripts and Jupyter notebooks were developed to handle the diverse datasets. These modules performed:
  - **Class Mapping:** Standardizing inconsistent class names across different datasets.
  - **Format Conversion:** Converting various annotation formats (e.g., Pascal VOC, Roboflow JSON, image masks) to the YOLO format.
  - **Image Enhancement:** Implementing functions for image straightening (using PCB masks) and basic color correction (clipping percentiles) to improve data quality.
  - **Tiling:** A custom implementation to split images into overlapping tiles to effectively enlarge small bounding boxes for better training.
- **Model Training Program:** Utilized the Ultralytics library in Python to train the YOLOv11 Nano models. This involved configuring training parameters, managing datasets, and executing the training loops.
- **UI Development:** The Flutter framework was used to build the desktop application, defining the layout, widgets, and user interaction flows for image upload, model selection, and result display. MobX was leveraged for reactive state management, ensuring efficient and responsive updates to the user interface as data changes.
- **Rust Backend Development:**
  - **FFI Integration:** `flutter_rust_bridge` was used to define the interfaces between Dart and Rust, automatically generating the necessary FFI code.
  - **Inference Engine:** The `ort` Rust library was integrated to load ONNX models and perform high-performance inference. This included setting up the ONNX Runtime session and managing input/output tensors.
  - **Image Processing:** Rust's `image` and `ndarray` libraries were used for efficient image loading, manipulation, and the implementation of the Slicing Aided Hyper Inference (SAHI) logic.

    - **Parallelism:** The `rayon` library was employed to parallelize computationally intensive tasks within the Rust backend, such as image slicing and batch inference.
- **Model Export:** Trained models were exported from their native format (e.g., PyTorch checkpoints from Ultralytics) to the ONNX format, making them compatible with the Rust `ort` inference engine.

5.3 Model Training

Two primary YOLOv11 Nano models were trained using the Ultralytics library, one for each specific detection task:

- **Copper Track Defect Detection Model:** This model was trained on the consolidated and preprocessed datasets containing various types of PCB track defects (e.g., short, spur, open, mouse bite, hole breakout, scratch, foreign objects). Iterative training, evaluation, and fine-tuning of hyperparameters were performed to optimize its performance across different defect classes.
- **Component Detection Model:** This model was trained on the aggregated and preprocessed datasets containing annotated images of various electronic components (e.g., battery, button, capacitor, IC, resistor, LED). The training process focused on enabling the model to accurately localize and classify these components, which is foundational for detecting missing components during the comparison phase in the UI.

During training, data augmentation techniques were applied to increase the diversity of the training data and improve model generalization. The models were iteratively refined based on their performance on validation datasets, aiming for optimal balance between precision and recall for their respective tasks.

CHAPTER 6: EVALUATION AND RESULTS

The evaluation phase was crucial for assessing the performance and robustness of both trained models. Each model underwent rigorous testing using various metrics and visualizations.

6.1 Evaluation Metrics Used

The following standard object detection evaluation metrics were used:

- **Precision:** The proportion of correct positive predictions out of all positive predictions made by the model.
- **Recall:** The proportion of actual positive instances that were correctly identified by the model.
- **F1-score:** The harmonic mean of precision and recall, providing a balanced measure of both.
- **Box-MAP (Mean Average Precision - IoU threshold range 0.5 to 0.95):** The average of Average Precisions calculated across multiple Intersection over Union (IoU) thresholds (from 0.5 to 0.95, in steps of 0.05) and all classes. This provides a comprehensive measure of performance across varying detection strictness.
- **Box-MAP50 (Mean Average Precision - IoU threshold 0.5):** The Mean Average Precision specifically calculated when a detected box is considered correct if its IoU with a ground truth box is greater than 0.5.
- **Box-MAP75 (Mean Average Precision - IoU threshold 0.75):** The Mean Average Precision specifically calculated when a detected box is considered correct if its IoU with a ground truth box is greater than 0.75.

In addition, the evaluation runs yielded several graphical representations and figures, which serve to unequivocally demonstrate the model's efficacy. These are detailed in the below:

- **Confusion Matrix:** A Confusion Matrix is a table that summarizes the performance of a classification model on a set of test data where the true values are known. It displays the number of correct and incorrect predictions made by the model, broken down by each class. The matrix allows for a detailed analysis of how well a classification model performs, distinguishing between True Positives, True Negatives, False Positives, and False Negatives.
- **Normalized Confusion Matrix:** A Normalized Confusion Matrix is a variation of the standard Confusion Matrix where the counts are replaced by proportions or percentages. This normalization can be done either over each row (representing recall for each true class) or over each column (representing precision for each predicted class), or over the entire dataset. In this case the normalization is over each column i.e. each value in a column in divided by the number of ground truth instances for each class, thus representing precision. Normalization helps in understanding the model's performance irrespective of class imbalance and provides a clearer visual comparison of misclassifications across different classes.
- **Precision-vs-Confidence Curve:** The Precision-vs-Confidence Curve illustrates the relationship between a model's precision and varying confidence (or probability) thresholds. Precision, which is the ratio of true positive predictions to the total positive predictions, tends to increase as the confidence threshold for classifying a positive instance is raised. This curve helps in selecting an optimal threshold that balances the

desire for high precision with the need to capture a sufficient number of positive instances.

- **Recall-vs-Confidence Curve:** The Recall-vs-Confidence Curve demonstrates how the recall of a model changes as the confidence threshold is adjusted. Recall, also known as sensitivity or true positive rate, is the proportion of actual positive cases that are correctly identified by the model. As the confidence threshold is lowered, the model becomes more lenient in its predictions, typically leading to higher recall but potentially at the cost of increased false positives. This curve is crucial for understanding the model's ability to identify all relevant instances.
- **F1-vs-Confidence Curve:** The F1-vs-Confidence Curve displays the F1-score at different confidence thresholds. The F1-score is the harmonic mean of precision and recall, providing a single metric that balances both. This curve is particularly useful when seeking a threshold that offers a robust trade-off between precision and recall, especially in scenarios where both false positives and false negatives carry significant costs. The peak of this curve often indicates the optimal operating point for a model.
- **Precision-vs-Recall Curve:** The Precision-vs-Recall Curve, often referred to as the PR curve, plots precision against recall for various classification thresholds. Unlike the Precision-vs-Confidence or Recall-vs-Confidence curves which use a direct confidence value, the PR curve shows the inherent trade-off between these two metrics without explicitly showing the threshold. It is particularly informative for imbalanced datasets, as a high area under the curve (AUC-PR) indicates good performance across different levels of recall.

6.2 Track Defect Detection Model Evaluation

The Track Defect Detection Model was evaluated in four distinct ways to provide a comprehensive understanding of its performance:

- On the entire dataset (full).
- On the entire dataset, treated as a single class (i.e., by combining all problem types into one).
- On the large PCB images dataset.
- On the large PCB images dataset, with a single class.

6.2.1 Results

The best model achieved the following evaluation output:

	Class Name	Precision	Recall	F1-score	Box-MAP	Box-MAP50	Box-MAP75
Full	**Short**	0.75861	0.63978	0.69415	0.34406	0.67146	0.29448
	Spur	0.76153	0.53222	0.62655	0.34406	0.67146	0.29448
	Spurious copper	0.61793	0.64026	0.62890	0.34406	0.67146	0.29448
	Open	0.70308	0.70270	0.70289	0.34406	0.67146	0.29448
	Mouse bite	0.69113	0.55098	0.61315	0.34406	0.67146	0.29448
	Hole breakout	0.81272	0.96760	0.88342	0.34406	0.67146	0.29448
	Conductor scratch	0.57191	0.55369	0.56265	0.34406	0.67146	0.29448
	Conductor foreign object	0.58090	0.48402	0.52805	0.34406	0.67146	0.29448
	Base material foreign object	0.67865	0.75600	0.71524	0.34406	0.67146	0.29448
	Missing hole	0.73747	0.41842	0.53391	0.34406	0.67146	0.29448
Full (Single Class)	**Combined**	0.77179	0.65834	0.71057	0.37722	0.74098	0.32281
Large PCBs	**Short**	0.43373	0.35714	0.39173	0.17780	0.39143	0.11074
	Spur	0.51660	0.23636	0.32433	0.17780	0.39143	0.11074
	Spurious copper	0.47770	0.42857	0.45181	0.17780	0.39143	0.11074
	Open	0.66475	0.43056	0.52262	0.17780	0.39143	0.11074
	Mouse bite	0.50888	0.40000	0.44792	0.17780	0.39143	0.11074
	Missing hole	0.59376	0.47629	0.52858	0.17780	0.39143	0.11074
Large PCBs (Single Class)	**Combined**	0.59917	0.42411	0.49666	0.19223	0.43753	0.11418

Table 6.2.1: Copper Track Defect Detection Model Results & Metrics

6.2.2 Confusion Matrix

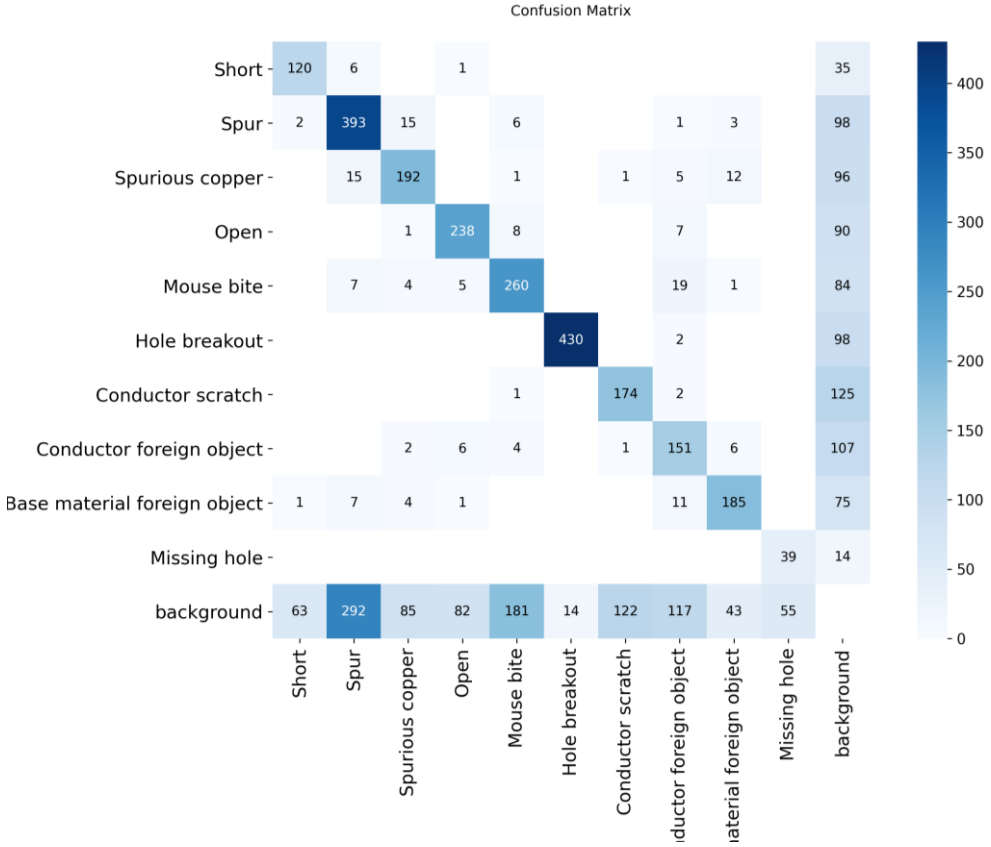


Figure 6.2.1 Track Defect Detection Model Confusion Matrix

6.2.3 Normalized Confusion Matrix

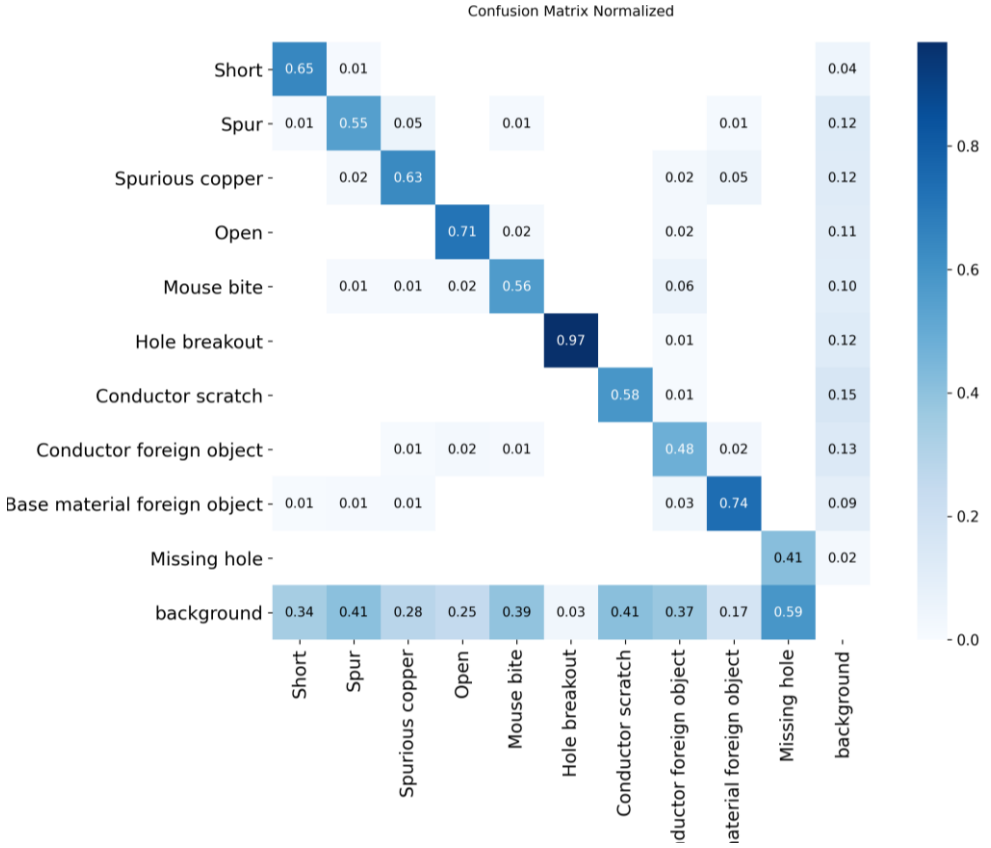


Figure 6.2.2 Track Defect Detection Model Normalized Confusion Matrix

6.2.4 Precision-vs-Confidence Curve

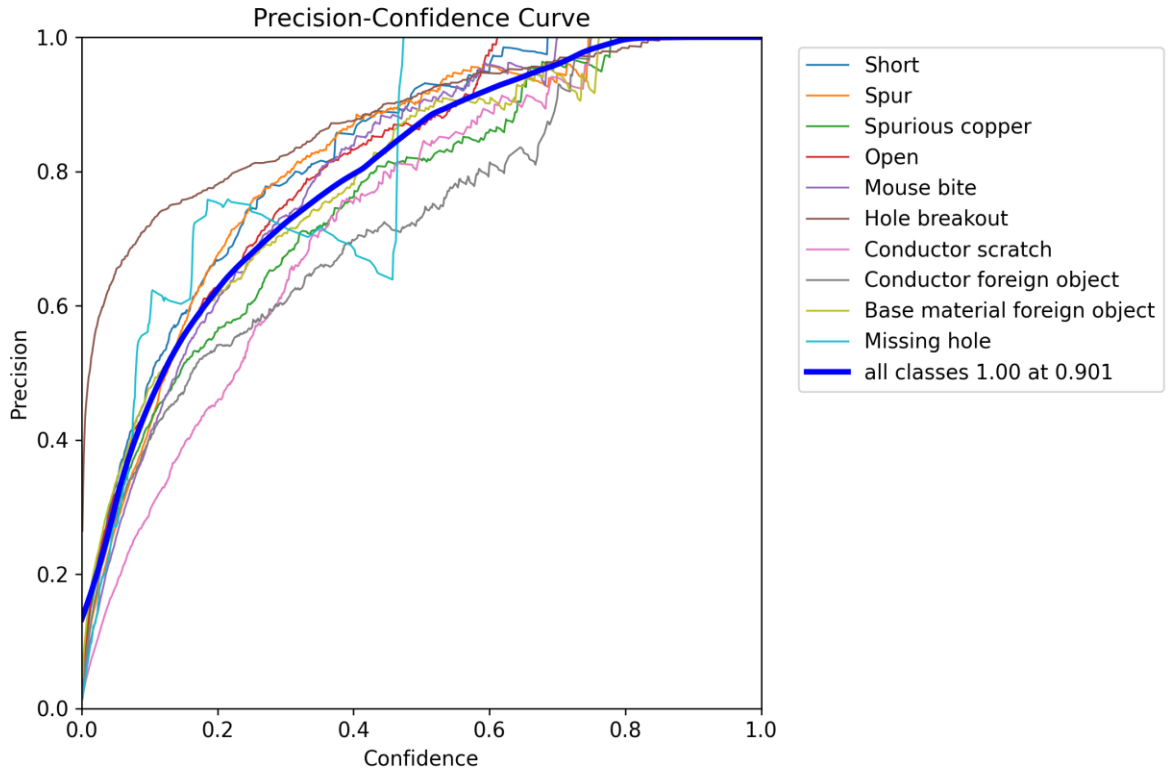


Figure 6.2.3 Track Defect Detection Model Precision-vs-Confidence Curve

6.2.5 Recall-vs-Confidence Curve

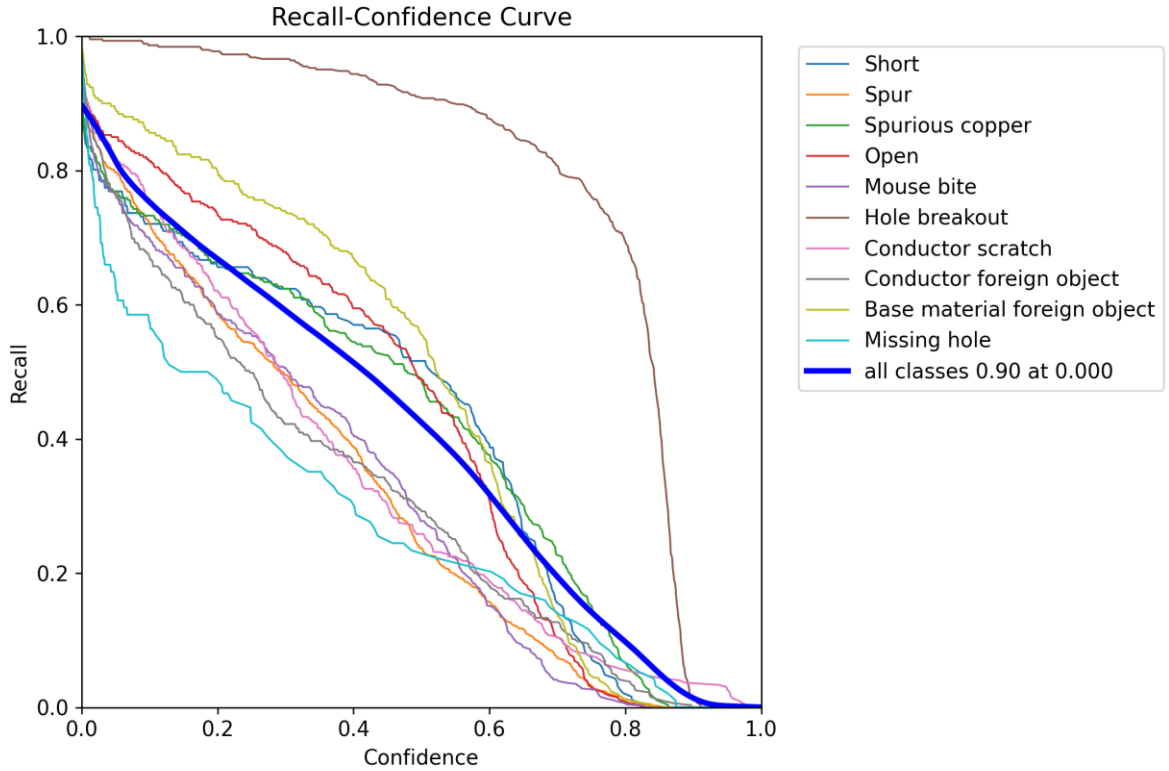


Figure 6.2.4 Track Defect Detection Model Recall-vs-Confidence Curve

6.2.6 F1-vs-Confidence Curve

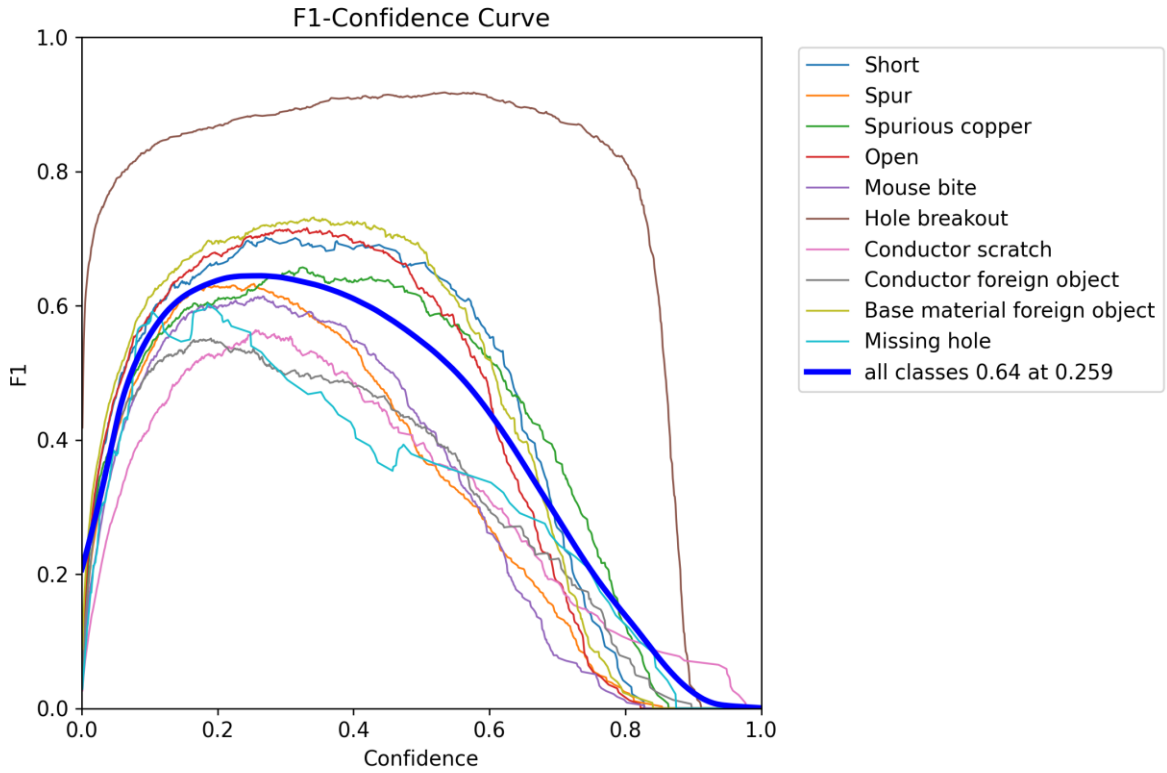


Figure 6.2.5 Track Defect Detection Model F1-vs-Confidence Curve

6.2.7 Precision-vs-Recall Curve

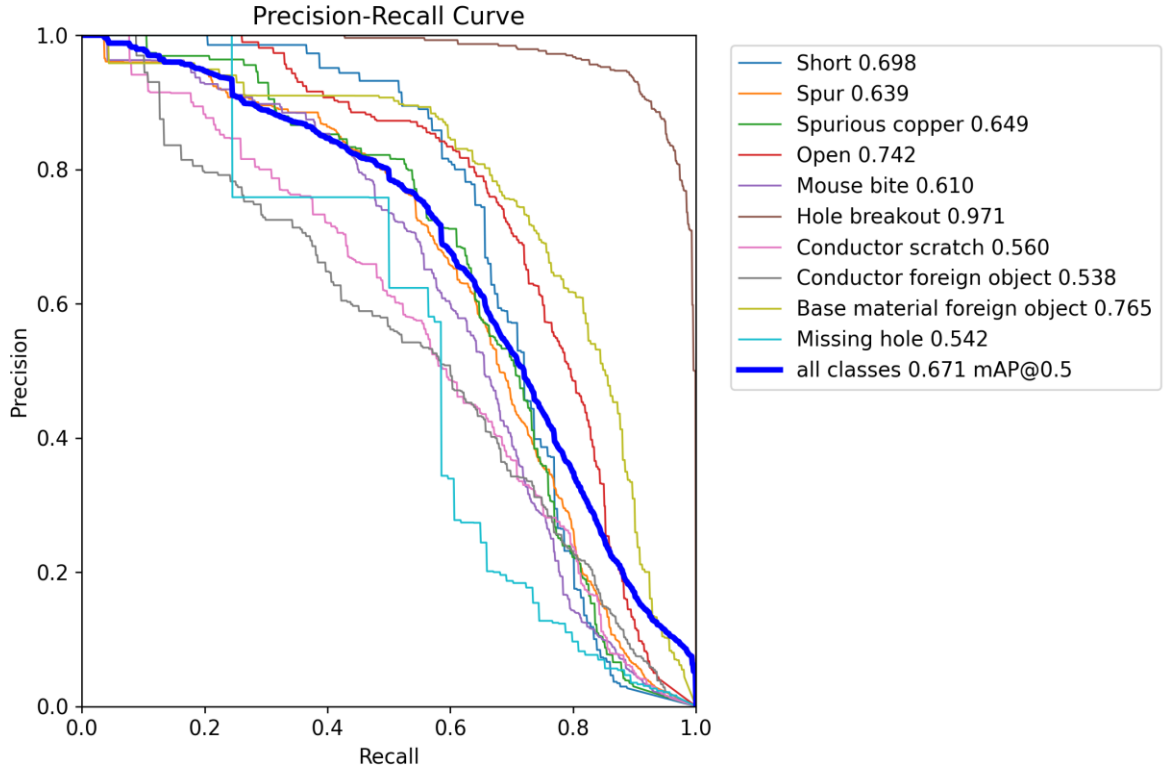


Figure 6.2.6 Track Defect Detection Model Precision-vs-Recall Curve

6.2.8 Confusion Matrix for the Large PCB Image dataset

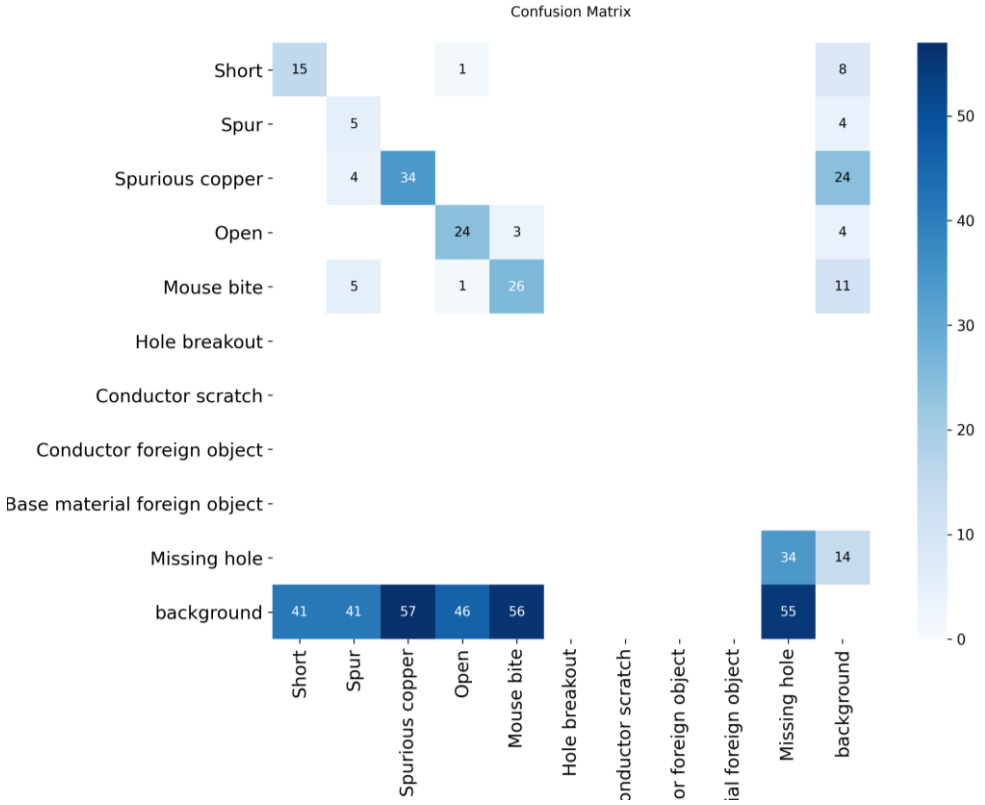


Figure 6.2.7 Track Defect Model Confusion Matrix for Large PCB Image dataset

6.2.9 Normalized Confusion Matrix for the Large PCB Image dataset

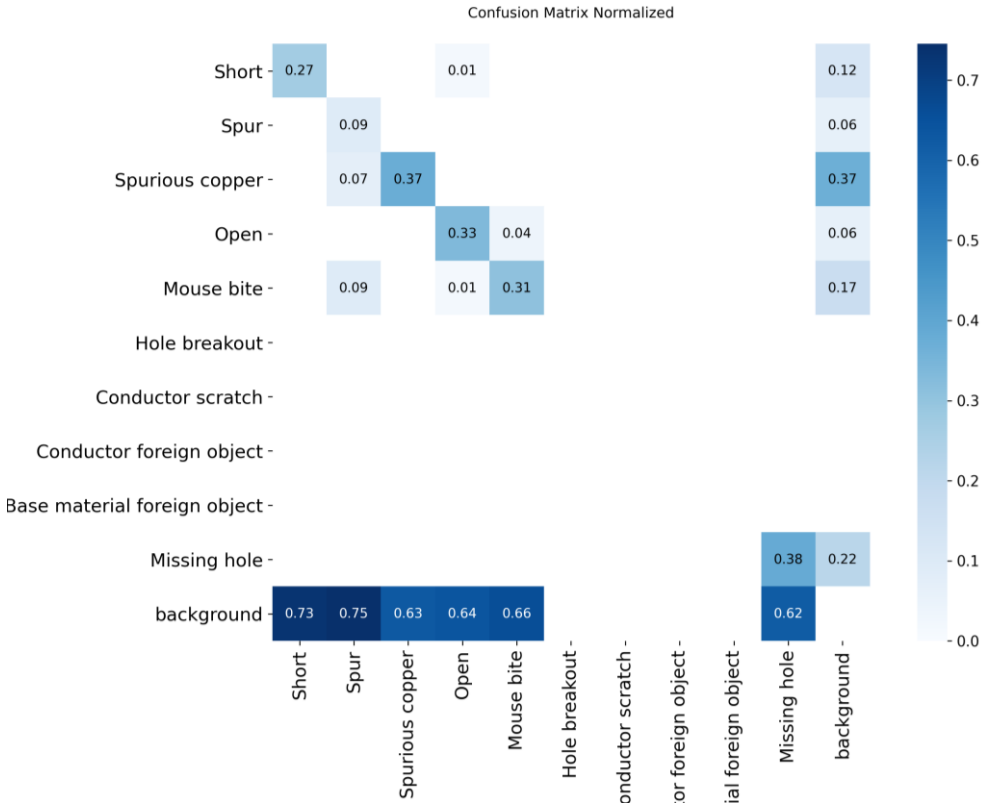


Figure 6.2.8 Track Defect Model Normalized Confusion Matrix for Large PCB Images

It is important to note that the Large PCB Image dataset does not encompass all classes present in the DsPCBSD+ dataset (and, by extension, the final PCB track dataset); consequently, certain fields within the confusion matrix remain unpopulated.

6.2.10 Precision-vs-Confidence Curve for the Large PCB Image dataset

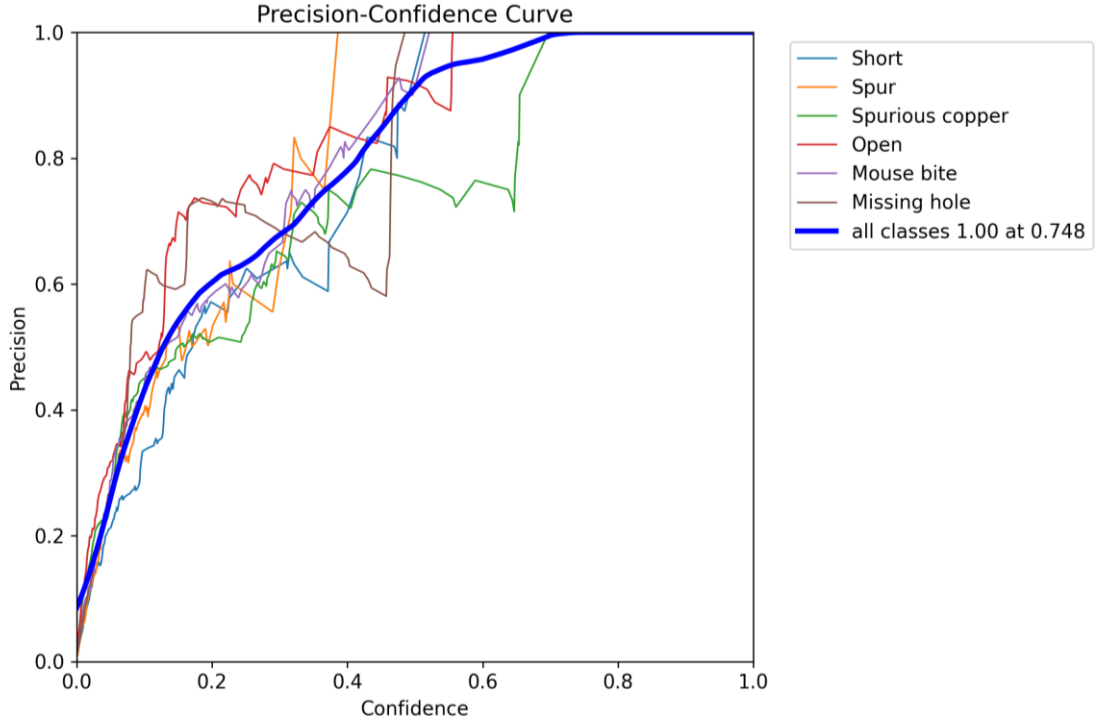


Figure 6.2.9 Track Defect Model Precision-vs-Confidence for Large PCB Image dataset

6.2.11 Recall-vs-Confidence Curve for the Large PCB Image dataset

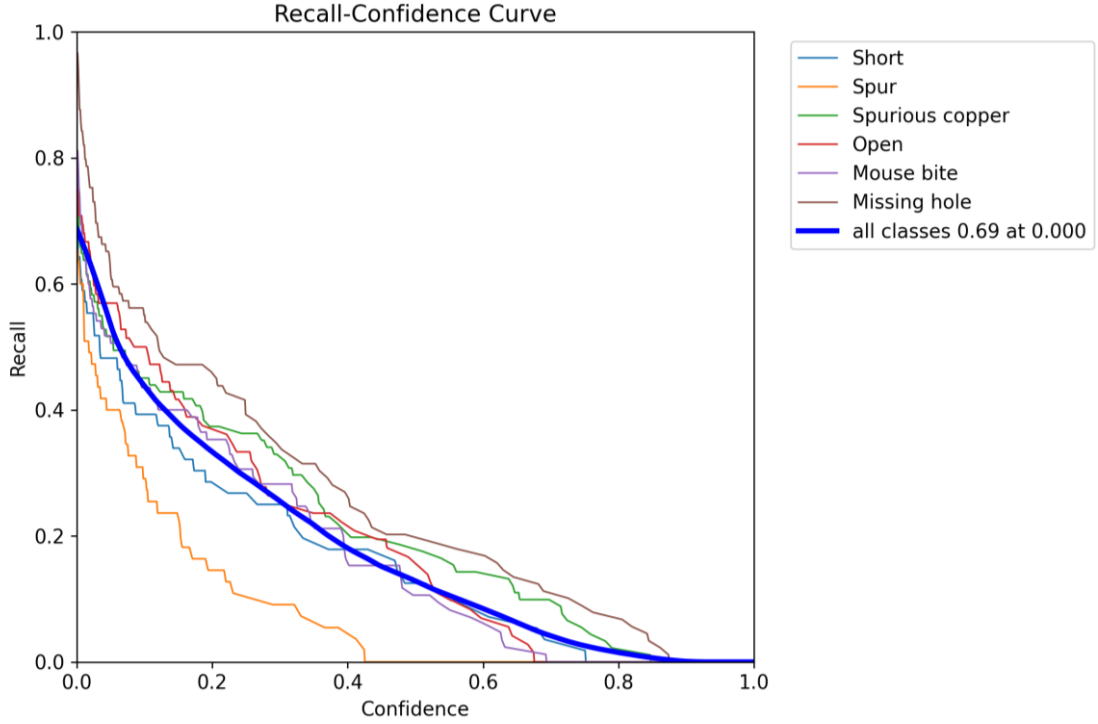


Figure 6.2.10 Track Defect Model Recall-vs-Confidence for Large PCB Image dataset

6.2.12 F1-vs-Confidence Curve for the Large PCB Image dataset

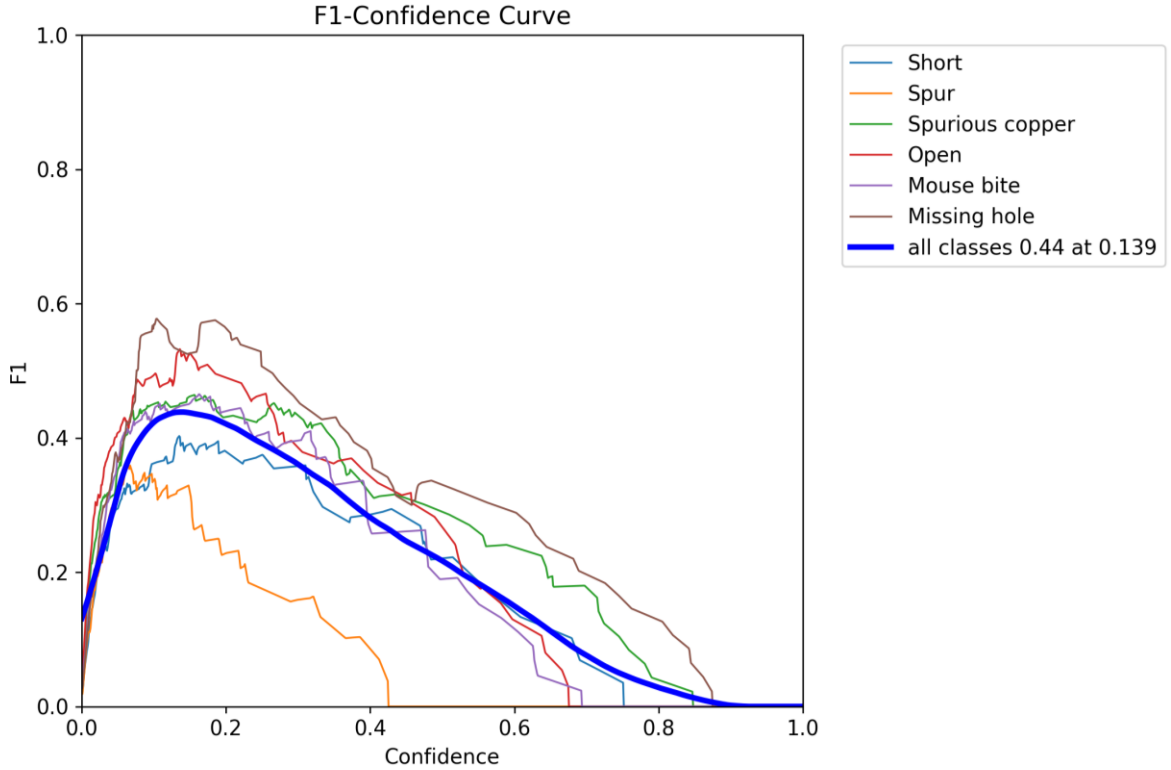


Figure 6.2.11 Track Defect Model F1-vs-Confidence for Large PCB Image dataset

6.2.13 Precision-vs-Recall Curve for the Large PCB Image dataset

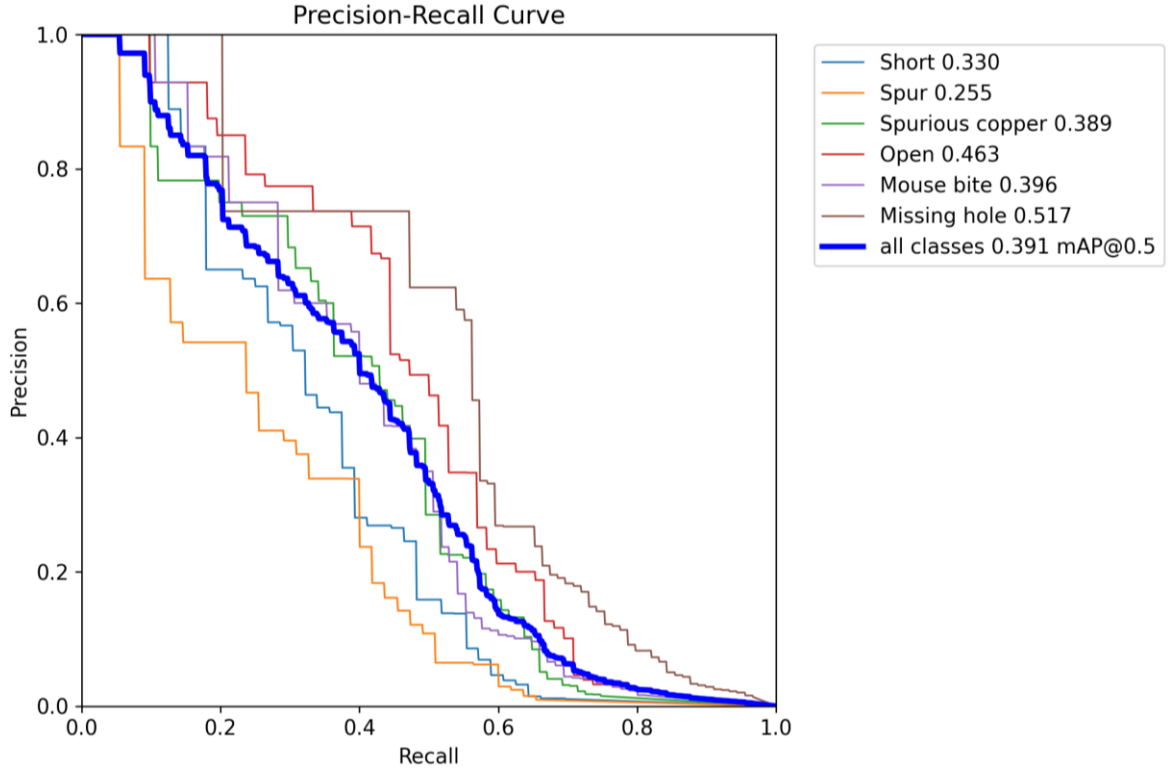


Figure 6.2.12 Track Defect Model Precision-vs-Recall for Large PCB Image dataset

The aggregated “Single Class” results are not presented here, because their performance curves are inherently represented by the average curve (as shown by the thick blue curve) within the preceding plots.

6.2.14 Summary of Track Defect Model Results

The best model for track defect detection demonstrates strong performance on the full dataset, with a notable F1-vs-Confidence curve that is wide and high, indicating robust performance across a broad range of confidence values.

While its confusion matrix shows high values, it occasionally produces false positives, particularly evident in the "background" row.

Performance on the large PCB dataset shows a slight decrease, as indicated by its F1-vs-Confidence curve. Though this problem is ultimately resolved by the Tiling process as described briefly before and then in detail later on in this report.

The optimal probability threshold for this model was determined to be 0.25 (25%), which will be used for final deployment. This model is designated as **Model CopperTrack**.

6.3 Component Detection Model Evaluation

The Component Detection Model was evaluated in two ways:

- On the entire dataset (full).
- On the entire dataset, with a single class (by combining all component types as the same).

6.3.1 Results

The best results for the Component Detection Model are as follows:

	Class Name	Precision	Recall	F1-score	Box-MAP	Box-MAP50	Box-MAP75
Full	**battery**	0.48056	0.71429	0.57457	0.38478	0.61576	0.41728
	button	0.88862	0.36275	0.51519	0.38478	0.61576	0.41728
	buzzer	0.91834	0.84615	0.88077	0.38478	0.61576	0.41728
	capacitor	0.85284	0.62589	0.72195	0.38478	0.61576	0.41728
	clock	0.60928	0.41667	0.49489	0.38478	0.61576	0.41728
	connector	0.60877	0.48309	0.53870	0.38478	0.61576	0.41728
	diode	0.68364	0.55175	0.61066	0.38478	0.61576	0.41728
	display	0.44909	0.70000	0.54715	0.38478	0.61576	0.41728
	fuse	0.59291	0.80000	0.68106	0.38478	0.61576	0.41728
	ic	0.71568	0.83037	0.76877	0.38478	0.61576	0.41728
	inductor	0.47205	0.39218	0.42842	0.38478	0.61576	0.41728
	led	0.68681	0.49583	0.57590	0.38478	0.61576	0.41728
	pads	0.31419	0.05019	0.08656	0.38478	0.61576	0.41728
	pins	0.21008	0.31959	0.25352	0.38478	0.61576	0.41728
	potentiometer	0.71059	0.40948	0.51956	0.38478	0.61576	0.41728
	relay	0.79690	1.00000	0.88697	0.38478	0.61576	0.41728
	resistor	0.86273	0.65675	0.74578	0.38478	0.61576	0.41728
	switch	0.87072	0.68504	0.76680	0.38478	0.61576	0.41728
	transistor	0.75751	0.66376	0.70754	0.38478	0.61576	0.41728
Full (Single Class)	**Combined**	0.84495	0.66326	0.74316	0.42653	0.73219	0.44990

Table 6.3.1: Component Detection Model Results & Metrics

6.3.2 Confusion Matrix

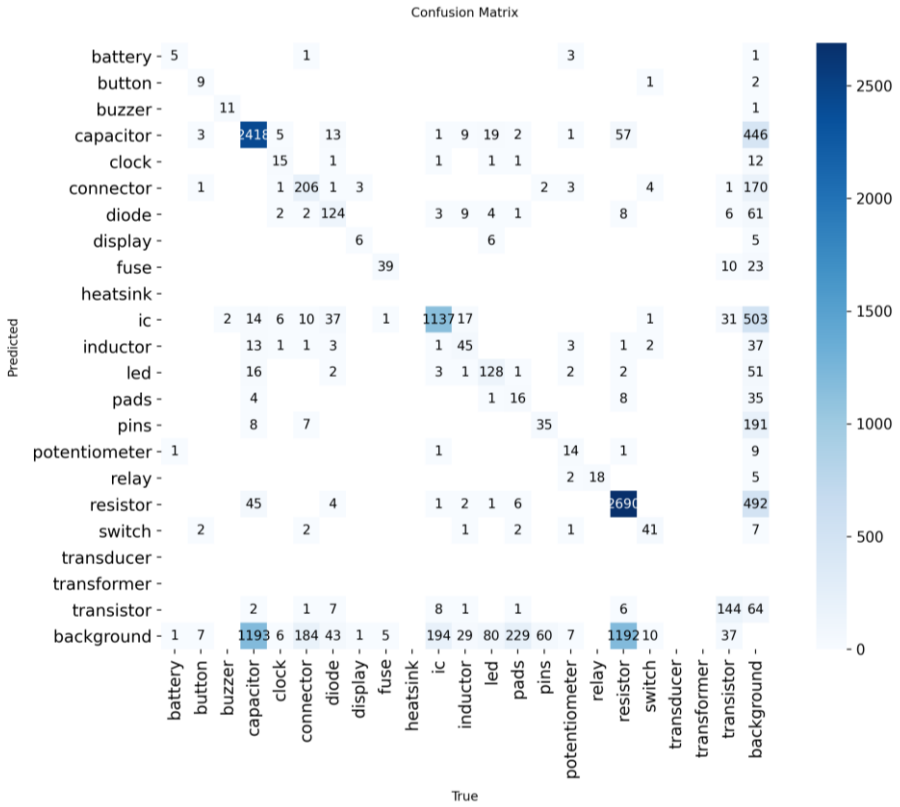


Figure 6.3.1 Component Detection Model Confusion Matrix

6.3.3 Normalized Confusion Matrix

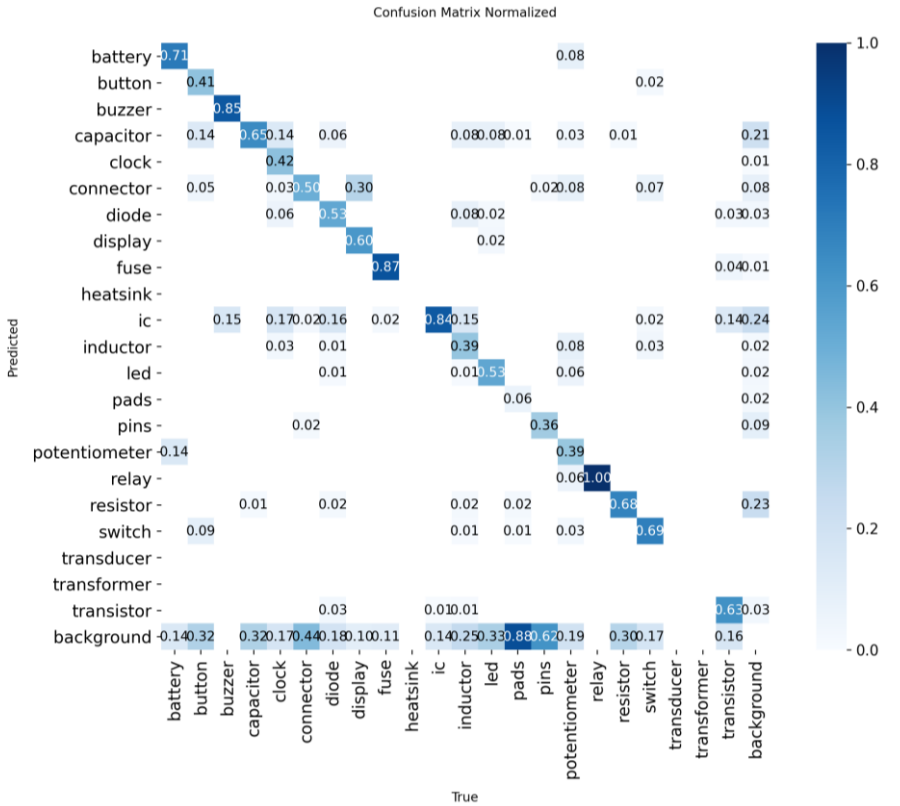


Figure 6.3.2 Component Detection Model Normalized Confusion Matrix

6.3.4 Precision-vs-Confidence Curve

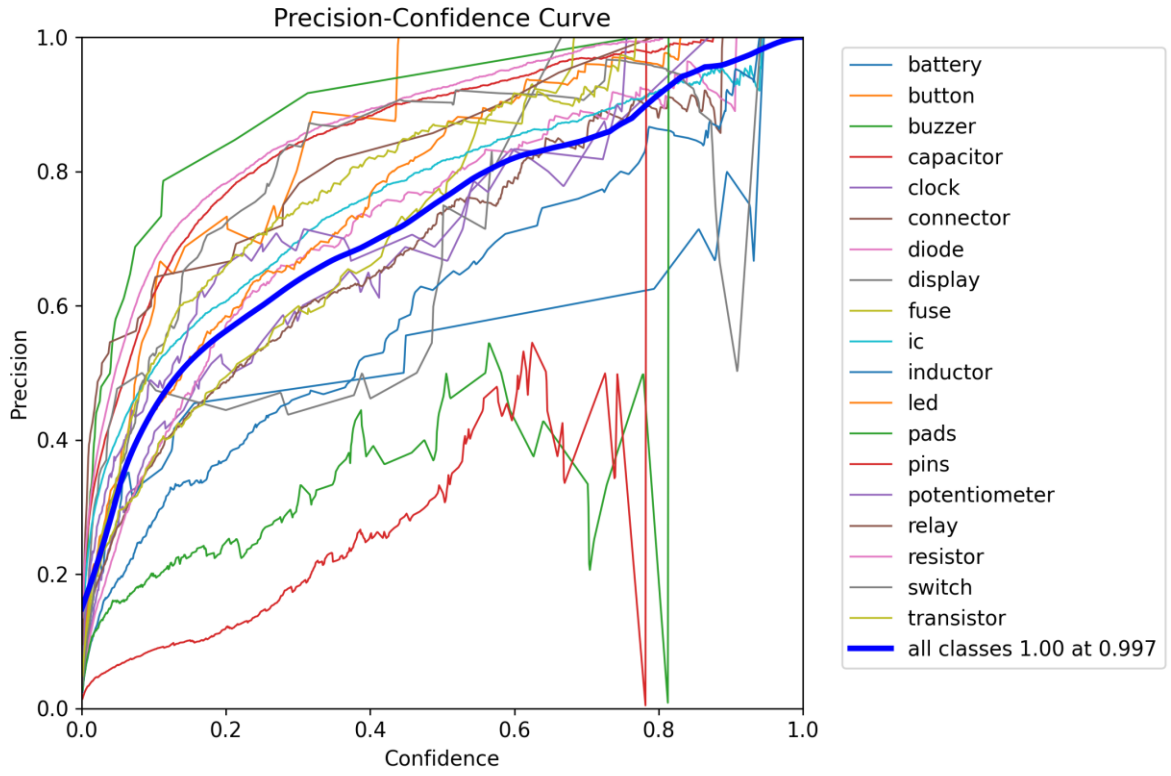


Figure 6.3.3 Component Detection Model Precision-vs-Confidence Curve

6.3.5 Recall-vs-Confidence Curve

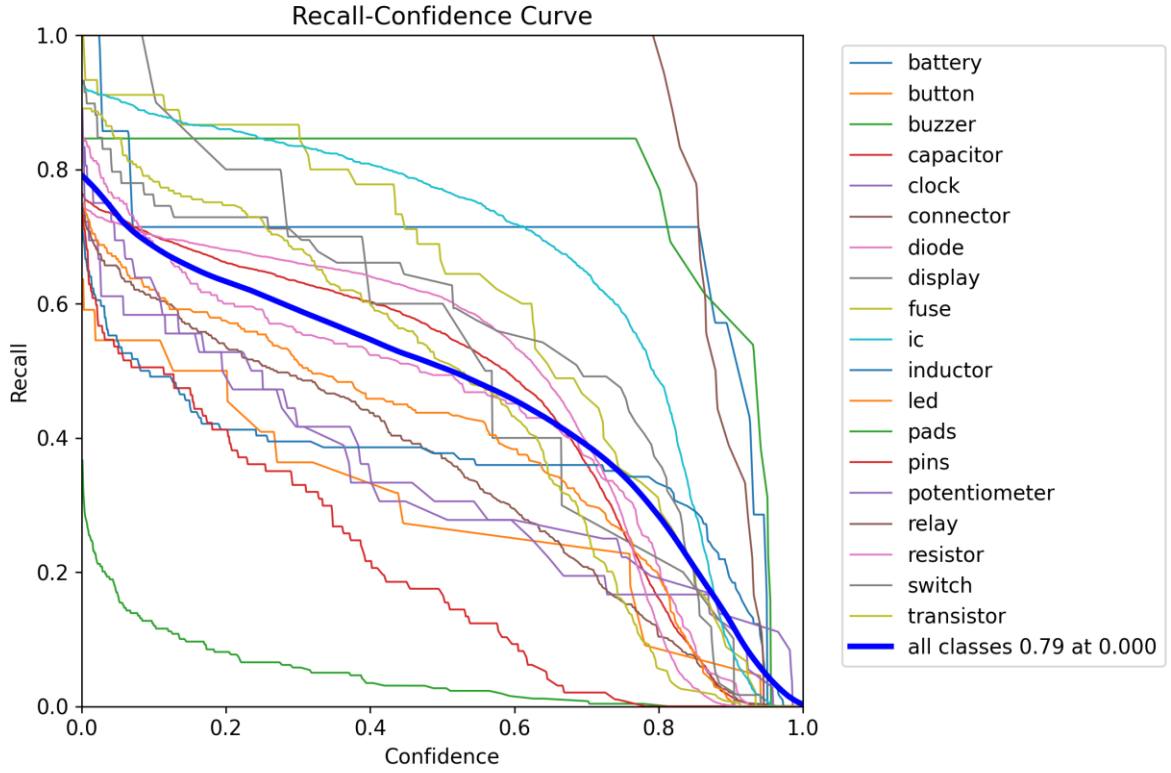


Figure 6.3.4 Component Detection Model Recall-vs-Confidence Curve

6.3.6 F1-vs-Confidence Curve

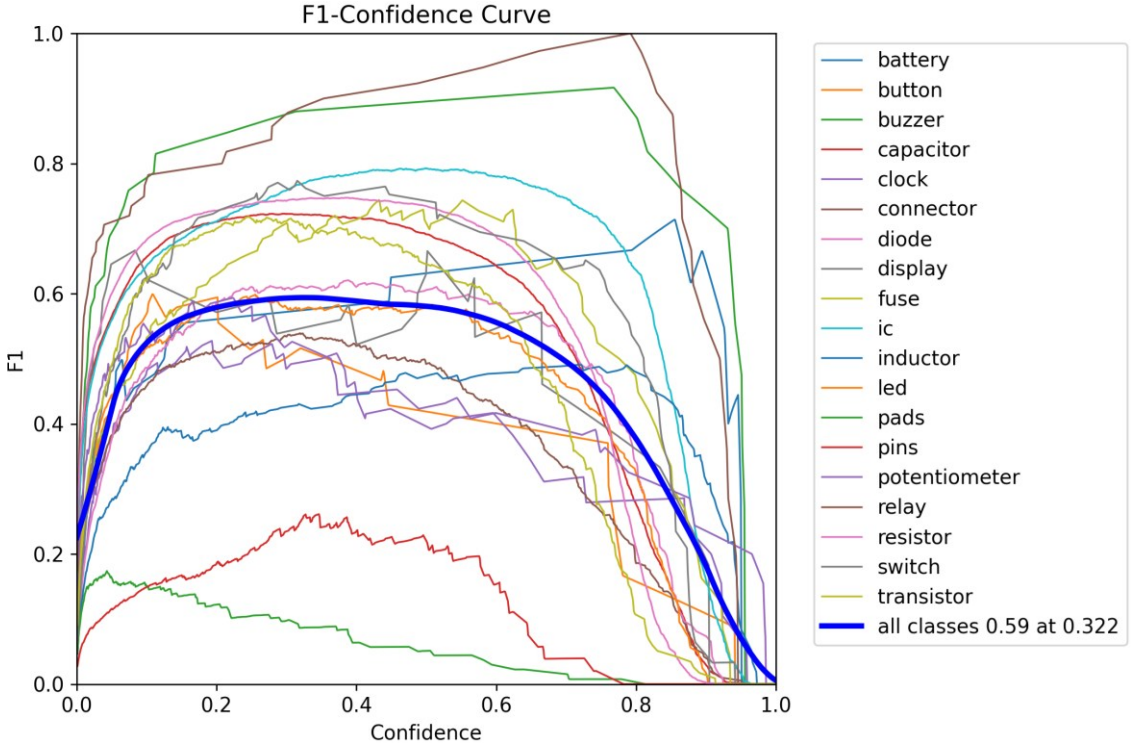


Figure 6.3.5 Component Detection Model F1-vs-Confidence Curve

6.3.7 Precision-vs-Recall Curve

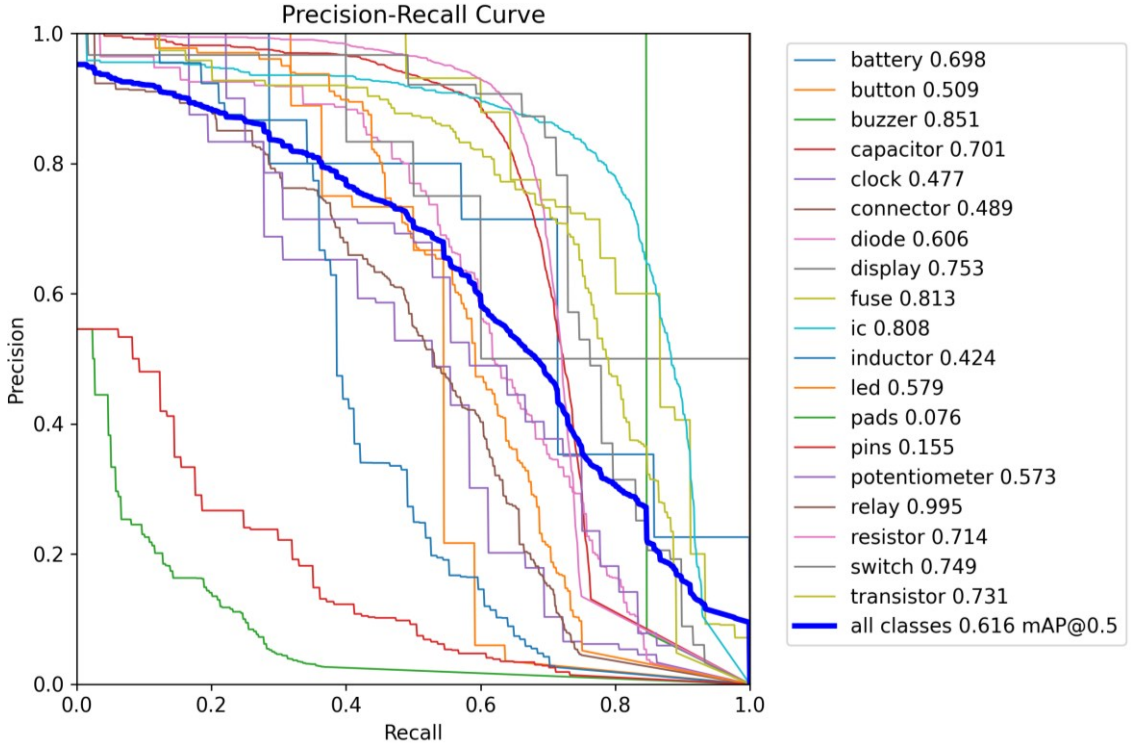


Figure 6.3.6 Component Detection Model Precision-vs-Recall Curve

The aggregated “Single Class” results are not presented here, as before, because their performance curves are inherently represented by the average curve (as shown by the thick blue curve) within the preceding plots.

6.3.8 Summary of Component Detection Model Results

The Component Detection Model also demonstrates strong performance. Its F1-vs-Confidence curve on the general dataset is very wide and high, indicating robust performance across a wide range of confidence values or probability thresholds.

The confusion matrix similarly shows high values, although it occasionally produces false positives, particularly observed in the "background" row.

The component detection model further demonstrates robust performance, evidenced by its fairly high Area Under the Precision-Recall Curve (AUC-PR), which signifies strong efficacy across varying recall thresholds.

The optimal probability threshold for this model is 0.322 (approximately 32%), which will be rounded down to 30% for final deployment.

6.4 Model Deployment

Both trained models were exported to the ONNX format for deployment. The ONNX format ensures compatibility with the Rust backend's ONNX Runtime (`ort`) library, facilitating efficient and portable inference.

The final deployed application is a Flutter desktop application, `pcb_fault_detection_ui`, which integrates these ONNX models for real-time PCB inspection.

A demonstration of the deployed application is available on YouTube.

6.5 Return on Investment (ROI)

This project delivers substantial Return on Investment (ROI) for electronics manufacturing companies by addressing critical operational and quality challenges. The key areas of impact include:

- **Reduced Labor Costs**: The automation of PCB inspection tasks directly diminishes the reliance on extensive manual labor, which is often tedious, prone to human error, and costly. By deploying the AI-driven system, manufacturers can reallocate skilled personnel to more complex or value-added tasks, significantly lowering operational expenditures associated with human inspection teams, including wages, training, and benefits.
- **Decreased Rework and Scrap**: The system's early and highly accurate detection of defects is paramount. By identifying flaws at an initial stage, faulty PCBs are prevented from advancing further into the manufacturing process, avoiding the compounding costs associated with subsequent assembly steps (e.g., adding expensive components to a defective board). This proactive identification dramatically reduces material waste (scrap) and minimizes the need for costly, time-consuming rework, thereby optimizing resource utilization.
- **Improved Product Quality**: Enhanced quality control, facilitated by consistent and objective AI-powered inspection, leads directly to a decrease in the number of defective products reaching the market. This translates into fewer field returns, warranty claims,

and customer complaints, which are major drains on resources and reputation. Consequently, customer satisfaction is bolstered, and the company's brand image and trustworthiness are significantly protected and enhanced in a competitive market.

- **Increased Throughput**: The superior speed of automated inspection, compared to manual methods, enables manufacturers to process a higher volume of PCBs within the same timeframe. This acceleration of inspection cycles directly contributes to increased production capacity and a faster time-to-market for products. The ability to push products through the manufacturing pipeline more quickly can provide a significant competitive advantage and facilitate responsiveness to market demands.

These synergistic qualitative benefits collectively culminate in a substantial positive ROI, primarily realized through enhanced operational efficiency, superior product quality, and strengthened market positioning.

CHAPTER 7: TESTING

7.1 Testing Plan / Strategy

The testing strategy for the PCB Fault Detection system primarily focused on the rigorous evaluation of the machine learning models. This involved:

- **Quantitative Evaluation:** Using standard object detection metrics (Precision, Recall, F1-score, mAP) to objectively measure model performance.
- **Qualitative Evaluation:** Analyzing confusion matrices and various performance curves (Precision-vs-Confidence, Recall-vs-Confidence, F1-vs-Confidence, Precision-vs-Recall) to understand model behavior across different confidence thresholds and identify class-specific strengths and weaknesses.
- **Dataset-Specific Evaluation:** Testing models on the full aggregated datasets and on specific subsets (e.g., large PCB images) to assess generalization capabilities.
- **Single-Class Evaluation:** Evaluating performance when all defect/component types are treated as a single class to understand overall detection capability.
- **Application-Level Testing:** The UI README elaborates on functional testing of the desktop application, ensuring proper image upload, model selection, slicing, inference, and visualization of results.

7.2 Test Results and Analysis

The detailed test results and their analysis are presented in Chapter 6 (Evaluation and Results):

- Both the Copper Track Defect Detection Model and the Component Detection Model demonstrated strong performance, with high F1-scores and mAP values on their respective datasets.
- Analysis of the confusion matrices revealed areas for potential improvement, such as occasional false positives in the "background" class for the track defect model.
- The performance curves provided insights into optimal confidence thresholds for deployment, ensuring a balance between precision and recall.

7.2.1 Test Cases

The evaluation metrics and visual results presented in Chapter 6 serve as evidence of the models' performance against various input conditions.

CHAPTER 8: CONCLUSION AND DISCUSSION

8.1 Key Takeaways: Overall Analysis of Project Viabilities

The AI-driven Optical PCB Inspection System demonstrates high viability as a solution for modern electronics manufacturing.

The successful development and integration of advanced machine learning models (YOLOv11), a high-performance Rust backend, and a user-friendly Flutter GUI prove that automated, accurate, and efficient PCB inspection is not only possible but also practical.

The project addresses a critical industry need by mitigating the shortcomings of manual inspection, offering a scalable and robust alternative that can significantly improve quality control and reduce operational costs.

8.2 Successes and Challenges: Problems Encountered and Solutions

During the project, several challenges were encountered, particularly during the data acquisition and preprocessing phases, and solutions were successfully implemented:

- **Inconsistent Data Formats and Labeling:** Datasets from various sources had differing annotation formats (Pascal VOC, Roboflow JSON, image masks) and inconsistent class naming.
    - **Solution:** Extensive preprocessing scripts and Jupyter notebooks were developed to standardize class labels through comprehensive mappings and convert all annotation formats to a unified YOLO format.
- **Image Quality and Orientation Issues:** Some datasets contained dark or tilted PCB images, which could negatively impact model training.
    - **Solution:** Custom image preprocessing steps were implemented, including image straightening (using PCB masks) and basic color correction (clipping pixel values), to normalize image quality and orientation.
- **Detection of Small Objects:** Many SMD components and fine track defects were represented by very small bounding boxes, posing a challenge for accurate detection.
    - **Solution:** A "Tiling" strategy was implemented during training to effectively enlarge small objects within cropped image tiles. For inference, Slicing Aided Hyper Inference (SAHI) was adopted, which slices images into overlapping sub-images to improve detection accuracy for small objects.
- **Application Binary Size for Deployment:** Including all possible ONNX Runtime execution providers (e.g., CUDA) significantly increased the application's size.
    - **Solution:** The `ort` library was compiled only with DirectML support (and CPU fallback), balancing GPU acceleration with a more manageable application size suitable for broader deployment on devices without high-end CUDA-compatible GPUs.

8.3 Summary of Project Work

This project successfully developed an AI-driven optical PCB inspection system, designed to enhance quality control in electronics manufacturing.

The system leverages a dual-model approach, employing two specialized YOLOv11 nano models for copper track defect detection and component detection, respectively.

A comprehensive data pipeline was established, involving acquisition from diverse sources and extensive preprocessing to standardize and enhance the datasets.

The evaluation demonstrated strong performance for both models, validating the system's capability to accurately identify PCB faults.

The user interface, built with Flutter, provides an intuitive experience for image upload and defect visualization, while a high-performance Rust backend handles efficient machine learning inference using ONNX models and advanced image processing techniques like SAHI.

8.4 Limitations and Future Enhancement

8.4.1 Limitations:

- **Execution Provider Scope:** The current deployed application's Rust backend is compiled with DirectML support and CPU fallback for ONNX Runtime. While this offers broad compatibility, it does not include other high-performance execution providers like CUDA, which could offer faster inference on specific NVIDIA GPU hardware.
- **Standalone Application:** The current system is designed as a standalone desktop application. It does not inherently include features for multi-user collaboration, cloud integration for data storage, or direct integration with manufacturing execution systems (MES) without further development.
- **Dataset Specificity:** While comprehensive, the models' performance is inherently tied to the diversity and quality of the training data. Performance on novel defect types or component variations not present in the training data might be less optimal.

8.4.2 Future Enhancements:

- **UI/UX (User Interface/User Experience) Improvements**:
    - Consider the potential addition of a feature for direct image upload from mobile devices. While this functionality would primarily serve demonstration purposes, practical applications would necessitate interfacing with a professional PCB image scanner, which typically connects to a desktop system.
    - Furthermore, incorporating synchronized pan and zoom capabilities for both benchmark and current test images could significantly enhance user interaction and analysis.
- **Expand Execution Provider Support:** Investigate dynamically loading or providing options for additional ONNX Runtime execution providers (e.g., CUDA, OpenVINO) to maximize performance on a wider range of hardware configurations.
- **Cloud Integration and Scalability:** Implement cloud-based solutions for model serving, data storage, and potentially a web-based interface to enable multi-user access and better scalability for large-scale manufacturing operations.
- **Automated Data Labeling/Active Learning:** Develop tools or integrate active learning strategies to reduce the manual effort required for annotating new defect types or expanding existing datasets.
- **Integration with Manufacturing Systems:** Develop APIs or connectors to integrate the system directly with existing MES or QMS platforms for a more seamless workflow within a smart factory environment.
- **Anomaly Detection:** Explore unsupervised or semi-supervised anomaly detection techniques to identify novel or previously unseen defect types that were not part of the training data.
- **3D Inspection:** Investigate the integration of 3D imaging techniques (e.g., structured light, X-ray) for more comprehensive inspection, especially for solder joint quality or hidden defects.

8.5 Project Specifics (URLs)

8.5.1 Project URL (Deployed Application)

The primary working project URL for the deployed desktop application is its GitHub Releases page, where binaries can be downloaded:

- **Application Releases:** https://github.com/aryan-programmer/pcb_fault_detection_ui/releases/tag/v0.1.0
- **Direct Application ZIP Download:** https://github.com/aryan-programmer/pcb_fault_detection_ui/releases/download/v0.1.0/pcb_fault_detection_ui.zip
- **YouTube Demo Link:** https://youtu.be/tCxNRT4C0cI

8.5.2 GitHub URLs

- **UI Repository:** https://github.com/aryan-programmer/pcb_fault_detection_ui
- **Model Training Repository:** https://github.com/aryan-programmer/pcb-fault-detection
- **Demo Images Directory (within model training repo):** https://github.com/aryan-programmer/pcb-fault-detection/tree/master/test_images

8.5.3 Notebook URL

The project utilized Jupyter Notebooks for data preprocessing and model training. The code for these notebooks is available within the `pcb-fault-detection` GitHub repository.

8.5.4 Uploaded Dataset on Kaggle

Given the intricate aggregation and preprocessing pipeline associated with the component detection dataset, the consolidated dataset has been deposited on Kaggle, accessible at the link given below. Its approximate size is 2.87 GB.

- **PCB Component Detection Consolidated Dataset:**

  https://www.kaggle.com/datasets/aryanstein/pcb-component-detection-consolidated-dataset

This action facilitates access for other researchers and practitioners who may benefit from this aggregated data for their respective PCB component detection initiatives.

It should be noted that the uploaded dataset intentionally excludes tiled images, as these are considered redundant and may not be universally required or desired; they can be generated on an as-needed basis. Furthermore, their inclusion would have augmented the dataset's size to approximately 5 GB.

8.5.5 Dataset URLs

- **DsPCBSD+:** (Ref. [03])
  - https://universe.roboflow.com/pcb-egrla/dspcbsd/dataset/1
  - https://www.nature.com/articles/s41597-024-03656-8
- **MIXED PCB DEFECT DETECTION:** (Ref. [04])
  - https://data.mendeley.com/datasets/fj4krvmrr5/1
  - https://data.mendeley.com/datasets/fj4krvmrr5/2
- **PCB_DATASET:** (Ref. [05])
  - https://www.kaggle.com/datasets/akhatova/pcb-defects
- **pcb-oriented-detection:**
  - https://www.kaggle.com/datasets/yuyi1005/pcb-oriented-detection
- **WACV pcb-component-detection:** (Ref. [06])
  - https://sites.google.com/view/chiawen-kuo/home/pcb-component-detection
- **FICS-PCB:** (Ref. [07])
  - https://www.researchgate.net/publication/344475848_FICS-PCB_A_Multi-Modal_Image_Dataset_for_Automated_Printed_Circuit_Board_Visual_Inspection
  - https://trust-hub.org/#/data/fics-pcb
  - https://universe.roboflow.com/erl-n2gvo/component-detection-caevk/
- **PCB-Vision:** (Ref. [08])
  - https://arxiv.org/pdf/2401.06528
  - https://zenodo.org/records/10617721
- **CompDetect Dataset:**
  - https://universe.roboflow.com/dataset-lmrsw/compdetect
  - https://universe.roboflow.com/dataset-lmrsw/compdetect/dataset/23
- **PCB defects.v2i.yolov11 (Demo Dataset):**
  - https://universe.roboflow.com/uni-4sdfm/pcb-defects/dataset/2

REFERENCES

[01] You Only Look Once: Unified, Real-Time Object Detection by Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi (arXiv:1506.02640)

[02] F. C. Akyon, S. Onur Altinuc and A. Temizel, "Slicing Aided Hyper Inference and Fine-Tuning for Small Object Detection," 2022 IEEE International Conference on Image Processing (ICIP), Bordeaux, France, 2022, pp. 966-970, doi: 10.1109/ICIP46576.2022.9897990.

[03] Lv, S., Ouyang, B., Deng, Z. *et al.* A dataset for deep learning based detection of printed circuit board surface defect. *Sci Data* **11**, 811 (2024). https://doi.org/10.1038/s41597-024-03656-8

[04] Ancha, Vinodkumar; Vaddi, Ramesh (2024), "MIXED PCB DEFECT DATASET", Mendeley Data, V1, doi: 10.17632/fj4krvmrr5.1

[05] Huang, Weibo, and Peng Wei. "A PCB dataset for defects detection and classification." arXiv preprint arXiv:1901.08204 (2019)

[06] Data-Efficient Graph Embedding Learning for PCB Component Detection by Kuo, Chia-Wen and Ashmore, Jacob and Huggins, David and Kira, Zsolt under 2019 IEEE Winter Conference on Applications of Computer Vision (WACV)

[07] Lu, Hangwei & Mehta, Dhwani & Paradis, Olivia & Asadizanjani, Navid & Tehranipoor, Mark & Woodard, Damon. (2020). FICS-PCB: A Multi-Modal Image Dataset for Automated Printed Circuit Board Visual Inspection.

[08] E. Arbash, M. Fuchs, B. Rasti, S. Lorenz, P. Ghamisi and R. Gloaguen, "PCB-Vision: A Multiscene RGB-Hyperspectral Benchmark Dataset of Printed Circuit Boards," in IEEE Sensors Journal, vol. 24, no. 10, pp. 17140-17158, 15 May15, 2024, doi: 10.1109/JSEN.2024.3380826.

[09] https://www.python.org/

[10] https://flutter.dev/

[11] https://dart.dev/

[12] https://www.rust-lang.org/

[13] https://pytorch.org/

[14] https://github.com/ultralytics/ultralytics

[15] https://developer.nvidia.com/cuda-toolkit

[16] https://github.com/fzyzcjy/flutter_rust_bridge

[17] https://ort.pyke.io/

[18] https://github.com/rayon-rs/rayon

[19] https://github.com/onnx/onnx

[20] https://pub.dev/packages/mobx

[21] https://github.com/obss/sahi